

Software Reliability

ANTHONY IANNINO
JOHN D. MUSA

*AT&T Bell Laboratories
Whippany, NJ 07981*

1.	Introduction	85
1.1	Importance	85
1.2	History	88
1.3	Basic Terms and Concepts	93
1.4	Relation to Hardware Reliability	100
2.	Modeling Software Reliability	101
2.1	General Model Characteristics	103
2.2	Selecting a Model	106
2.3	Execution-Time Modeling	111
2.4	Calendar-Time Modeling.	120
2.5	Selected Model	135
3.	Application of Software Reliability	139
3.1	General Computations	140
3.2	System Engineering.	145
3.3	Project Management	152
3.4	Managing the Operational Phase	155
3.5	Evaluating Software Engineering Technologies	160
3.6	Documentation Debugging and Other Applications	161
4.	State of the Art, Future Development, and Summary	162
4.1	Measurement.	162
4.2	Prediction Quality and its Improvement	162
4.3	Possible Extensions.	166
	References	168

1. Introduction

1.1 Importance

The information industry, of primary significance in the world economy today and in the foreseeable future, continues to expand at a rapid rate. This expansion is partly related to the increase in cost-effectiveness of computer hardware. Cost-effectiveness has increased by a factor of about 1000 every decade (Musa, 1985). While this rate of change continues, the range of tasks

that can be handled most economically by computing will probably grow rapidly. Software is the major part of most computer systems, and hence the field of software engineering can expect similar rapid growth.

The principal forces affecting software engineering today include

1. greater and increasingly international business competition,
2. higher cost of both information system development and information system failure,
3. faster pace of change in computing technology, and
4. increasing complexity of managing information system development.

Most information system customers are businesses. The greater business competition they face sensitizes them acutely to their software product needs. At the same time, they are aware of greater competition among software producers so that they have become increasingly sophisticated and demanding. Software producers must understand their needs thoroughly and precisely. Three of the most significant needs are level of quality required, delivery date, and cost.

Both the development and operational costs of software are increasing, as the result of greater size, complexity, and degree of distribution of systems. There are many multicomputer systems linked by networks. They may have many diverse pieces of software that run simultaneously and interact. A growing proportion of systems operate in real time. Consequently, development costs are becoming higher. The heavy dependence of institutions on information systems, due to burgeoning applications, means that the operational effects of failure are large and often critical. For example, consider the effects of breakdowns of airline reservations, banking, medical monitoring, military defense, and nuclear power plant safety control systems. Costs of failure include not only direct expenses but also product liability risks and damage to a company's reputation. Dramatic impact on a company's market share and profitability can result.

Faster change in computing technology has made the time of delivery of a system increasingly crucial. The market window, or time period available for the introduction of a new product before that product is surpassed in capability or cost by another, has shortened.

These cost and schedule pressures have made it impossible to create a software product that simultaneously provides high quality, rapid delivery, and low cost. It would be nice to have programs that are "correct," but in the real world one must settle for something less. The tradeoffs must be carefully made to meet customer needs. This means that measurement and prediction of software product characteristics is essential.

Management of information system development has become more complex with increasing size and complexity of the systems. Many systems

now are divided into components that are developed by different companies. A definite managerial, contractual, and legal need exists for the clear identification of the characteristics of the system and its components and for indicators of progress during development. Again, the conclusion is that measurement and prediction of software product characteristics and the current status of those characteristics during development has become increasingly important.

Quality, delivery date, and cost are primarily customer-oriented rather than developer-oriented attributes. Quantitative measures exist for the latter two characteristics, but quantification of quality has been difficult. It is most important, however, because absence of a concrete measure for software quality generally means that quality will suffer in competing for attention against cost and delivery date. Indeed, this absence may be the principal reason for the poor quality of many software products. Software reliability is probably the most important of the characteristics inherent in the concept of "software quality" (Ramamoorthy *et al.*, 1987). It concerns itself with how well the software *functions* to meet the requirements of the customer. It is the probability that the software works without failure for a specified period of time. "Failure" means that an executing program does not meet customer requirements in some way. "Not functioning to meet customer requirements" is a very broad definition. Thus reliability includes many properties that are often quoted as aspects of quality, such as correctness and program *behavior* that is not user-friendly. Properties such as software safety are actually specialized aspects of software reliability. Two of the few aspects of quality that probably cannot be directly related to reliability are modifiability and understandability of documentation. In addition to its preeminent importance, software reliability has been the most readily quantifiable attribute of software quality.

Reliability is a customer-oriented view of software quality. Initial (and many present) approaches to measuring software quality were based on trying to count the faults or defects found in a program. This approach is developer-oriented. Projects generally counted either failures (the occurrences of malfunction) or repairs (for example, maintenance or correction reports), neither of which are equivalent to faults. Even if faults found are correctly counted, they are not a good status indicator (is a large number good or bad)? Faults remaining may be a better indicator. But faults found can only be used to make approximate comparisons with other projects, commonly made in terms of faults per 1000 developed source lines of code. Reliability is a much richer measure. It relates to operation rather than design of the program, and hence it is dynamic rather than static (Saunier, 1983). It accounts for the frequency with which faults cause problems. Reliability is easily associated with costs. It is more suitable than faults for examining the significance of trends, for setting objectives, and for predicting when those objectives will be

met. One can analyze in common terms the effect on system quality of software and hardware, both of which are present in any real system. Thus, reliability measures are much more useful than fault measures.

Some attention to faults still makes sense, but it should be focused on faults as predictors of reliability and on the nature of faults. A better understanding of faults and the causative human error processes should lead to strategies to avoid, detect (and hence remove), or compensate for them.

Measuring and predicting software reliability has become vital for software managers, software engineers, managers and engineers of products that include software, and to users of these products. A technology has been developed to help meet this need. Much practical knowledge has been gathered by applying it to actual projects. Some problems remain. Further progress in solving these problems necessitates confronting them in real project situations and gathering appropriate data. The technology is sufficiently accurate and useful that the benefits of use exceed the costs (Musa *et al.*, 1987).

Experience with the application of software reliability measurement and prediction on projects shows implementation cost of around 0.1 to 0.2% of project development costs. This includes the cost of education, the cost of data collection and processing, and the cost of studies. The cost figures assume that the project has an existing formal mechanism for reporting troubles.

Better decisions can save money during the software life cycle in many ways, some of which are discussed in Section 3. Usually the total savings will be more than 10 times greater than the cost of applying software reliability measurement. Consider the fairly typical example of a two-year software project with a six-month system test period. The amount of testing, although not the only factor, is generally highly correlated with reliability. Suppose a study shows that only five months of testing is needed to get the reliability required for the application. Then one might well be able to save 4% of the development cost of the project, about 20 to 40 times the cost of using measurement.

1.2 History

This section presents the historical development of software reliability models. The development through 1989 can be characterized by the following overlapping stages, events, or themes:

1. the creation of various models relating reliability to time, failures experienced, and other variables,
2. the discovery that measurement with respect to execution time substantially simplifies models,
3. the clarification of the distinction between "fault" and "failure" and the development of a rich conceptual base that amplifies the significance of this distinction,

4. the study of model parameter estimation,
5. the comparison of models and the development of comparison criteria,
6. the classification of models,
7. a focus on collecting better data,
8. the development of a "lore" of techniques in using and adapting models for the particular circumstances of various applications, and
9. the transition from use of mean-time-to-failure to failure intensity for characterizing status.

This section concentrates mainly on the first stage, but it also touches on the others. Related developments are grouped, resulting in deviations from strict chronology. For an interesting perspective by one of the pioneers, see Shooman (1984).

The first known study of software reliability (Hudson, 1967) viewed software development as a birth-and-death process (a type of Markov process). Fault generation (through design changes, faults created in fixing other faults, etc.) was a birth, and fault correction was a death. The number of faults remaining defined the process state. The transition probabilities characterized the birth and death functions. Hudson's analysis, limited to pure death processes for reasons of mathematical tractability, assumed that the rate of detection of faults was proportional to the number of faults remaining and a positive power of the time. Hudson showed that the number of faults detected follows a binomial distribution whose mean value has the form of a Weibull function of time. Data from the system test phase of one program was presented. Reasonable agreement between model and data was obtained when the system test phase was split into three overlapping subphases, with separate fits made for each.

The next major steps were made by Jelinski and Moranda (1972) and Shooman (1972). Both assumed a piecewise-constant hazard rate for failures that was proportional to the number of faults remaining. The hazard rate changes at each fault correction by a constant amount, but is unchanged between corrections. Jelinski and Moranda used maximum-likelihood estimation to determine the total number of faults in the software and the constant of proportionality between the number of faults remaining and the hazard rate. Shooman postulated that the hazard rate was proportional to the fault density per instruction, the number of unique instructions executed per unit time, and a bulk constant. The bulk constant represented the proportion of faults that cause failures. The fault density is the difference between the original fault density and the faults corrected per instruction. The functional form of the latter quantity as a function of time was assumed to be related to the project staffing level in time. Several different fault correction profiles were proposed. The choice would depend on the particular project one was working with.

Schick and Wolverton (1973) proposed another early model, with the hazard rate proportional to the product of the number of faults remaining and the time. Hence the size of the changes in hazard rate (at fault correction) increases with time. Wagoner (1973) suggested a model in which the hazard rate was proportional to the number of faults remaining and a power of the time. This power could be varied to fit the data.

Schick and Wolverton (1978) also proposed a modified model where the hazard rate was a parabolic function instead of a linear function in time. This hazard rate is close to, but not the same as, that of the Weibull distribution.

Schneidewind (1972) initially approached software reliability modeling empirically. He recommended the investigation of different reliability functions and selection of the distribution that best fit the particular project in question. Candidates were the exponential, normal, gamma, and Weibull distributions. In looking at data, Schneidewind found that the best distribution varied from project to project. He indicated the importance of determining confidence intervals for the parameters estimated rather than just relying on point estimates.

In a later paper (Schneidewind, 1975) he viewed fault detections per time interval as a nonhomogeneous Poisson process with an exponential mean-value function. He applied either least-squares or maximum-likelihood estimation to the determination of the parameters of the process. Schneidewind also suggested that the time lag between failure detection and correction be determined from actual data and used to correct the time scale in forecasts.

Moranda (1975) has also proposed two variants of his original model. The "geometric de-eutrophication process," has a hazard rate that decreases in steps that form a geometric progression. The second, called the "geometric Poisson" model, has a hazard rate that also decreases in a geometric progression. However, the decrements occur at fixed intervals rather than at each failure correction.

Shortly after some of this early work, Musa (1975) presented an execution-time model of software reliability (referred to as the "basic execution-time model" here). He postulated that execution time, the processor time utilized in executing the program, was the best practical measure of failure-inducing stress that was being placed on the program. Calendar time does not account for varying usage of the program. The removal of this confounding factor greatly simplifies modeling and yields better predictive validity.

The basic execution-time model exploits the fact that the fault correction rate is generally proportional to the fault detection or hazard rate, making consideration of debugging personnel profiles unnecessary. The model was tested in real time on four development projects with excellent results. The variability in models from project to project noted by Schneidewind and

Shooman did not occur. Hecht (1981) has independently verified the simplification resulting from the use of execution time.

A calendar-time component was developed for the model that relates execution time to calendar time, allowing execution-time predictions to be converted to dates. It is based on the fact that available resources limit the amount of execution time that is practical each calendar day.

Littlewood and Verrall (1973) developed a Bayesian approach to software reliability measurement. It views software reliability as a measure of strength of belief that a program will operate successfully. This contrasts with the classical "experimental" view of reliability. Almost all published models assume that failures occur randomly during the operation of the program. However, while most postulate that the value of the hazard rate is simply a function of the number of faults remaining, Littlewood and Verrall modeled it as a *random variable*. The distribution function of this random variable has a parameter that varies with the number of failures experienced and hence characterizes reliability change. Littlewood and Verrall proposed various functional forms for the description of this variation. In their procedure, the values of the parameters of each functional form that produce the best fit for that form are determined. Then the functional forms are compared (at the optimum values of the parameters) and the best fitting form is selected.

The concept of the hazard rate as a random variable reflects uncertainty in the effectiveness of fault correction. Note that the hazard rate changes discontinuously at each failure detection and correction and varies continuously with the cumulative execution time. The heights of the discontinuities usually but not necessarily decrease with failures experienced. The continuous decrease of hazard rate with time reflects one's subjective ideas of how reliability should change in a failure-free environment.

Keiller *et al.* (1983) investigated a model similar to the Littlewood and Verrall model. It characterizes the randomness of the hazard rate with the same distribution but uses a different parameter to express reliability change.

The differential fault model proposed by Littlewood (1981) may be viewed as a variant of the Littlewood and Verrall model. It also views the hazard rate as a random variable and uses Bayesian inference. Reliability growth is modeled through two quantities, the number of faults remaining and the variation of the per-fault hazard rate with time. This variation happens if failures occur with different frequencies due to the difference in frequency with which different input states of the program are executed. The most frequently occurring faults are detected and corrected first. Littlewood considers that uncertainties in reliability growth probably result more from uncertainties in the relative frequencies of execution of different input states than uncertainties in fault correction.

Goel and Okumoto (1978) developed a modification of the Jelinski and

Moranda model that accounts for imperfect debugging. The view of debugging taken here is that of a Markov process, with appropriate transition probabilities between states. Several useful quantities can be derived analytically, with the mathematics remaining tractable. Kremer (1983) developed this idea further, including the possibility of spawning new faults due to the repair activity.

In another paper, Goel and Okumoto (1979a) depicted software failure as a nonhomogeneous Poisson process with an exponentially decaying rate function. Maximum-likelihood methods for estimating the parameters were developed for two different situations: intervals between failures and failures per interval. A simple modification of this model was investigated by Yamada *et al.* (1983), where the cumulative number of failures detected is described as an S-shaped curve.

Crow (1974) proposed that a hardware model, based on a nonhomogeneous Poisson process with a failure intensity function that is a power function in time, be applied to software, using appropriate ranges of parameter values.

Thus, much of the early history of software reliability modeling consisted of looking at different possible models. In the late 1970s and early 1980s, efforts started to focus on comparing software reliability models, with the objective of selecting the "best" one or ones. Initial efforts at comparison (Sukert, 1979; Schick and Wolverton, 1978) suffered from lack of good failure data and lack of agreement on the criteria to be used in making the comparisons. The publication by Musa (1979a) of reasonably good-quality sets of data stimulated comparison efforts. Iannino *et al.* (1984) worked out a consensus on the comparison criteria to be employed. Examination of the basic concepts underlying software reliability modeling and development of a classification scheme (Musa and Okumoto, 1983) helped clarify and organize comparisons and suggested possible new models. This effort led to the development of the Musa and Okumoto logarithmic Poisson execution-time model (Musa and Okumoto, 1984a).

The logarithmic Poisson model is based on a nonhomogeneous Poisson process with an intensity function that decreases exponentially with expected failures experienced. It reflects the situation where failures repaired in an early phase of execution reduce the failure intensity more than those in a later stage. It is called "logarithmic" because the expected number of failures is a logarithmic function of time. Validation of the model with actual data indicated that it combined high predictive validity with great simplicity. The calendar-time component of this model is analogous to that of the basic execution-time model.

Interest in clarifying the distinction between "fault" and "failure" and developing the conceptual base for software reliability is relatively recent. Much of this material appears for the first time in the book by Musa *et al.*

(1987). Practical experience obtained from application, scattered through papers too numerous to list, has also been collected and organized in this book.

1.3 Basic Terms and Concepts

A *software failure* is a deviation of the external results of program operation from requirements. The program has to be executing for a failure to occur. Note that a failure is not the same thing as a “bug” or, more properly, “fault.” There are many kinds of failures, ranging from cessation of operation to minor discrepancies to deficiency in performance such as excessive response time.

A fault is a defect in the program that, when executed with certain input data, causes a failure. A fault may be the source of multiple failures. These may arise from repetition of the same input conditions, or executing different input data. A fault is a property of the program rather than a property of its execution or behavior. A fault is created when a programmer makes an *error*. It is *very* important to make the failure-fault distinction! There has been, and frequently remains, much confusion because these terms were mixed up.

To illustrate these terms, suppose that a user requests a display at program startup. It does not appear. That departure from how the program should behave is a failure. The fault associated with the failure might be that an initialization instruction for a variable was not coded. A fault does not have to be localized in nature; for example, it could be an inefficient routine that has to be rewritten.

Reliability quantities are usually defined in terms of time, but it's possible to use other variables. Three kinds of time are relevant to software reliability. The *execution time* for a program is the time required by a processor to execute the instructions of that program. *Calendar time* is the “regular” time we are familiar with. *Clock time*, used occasionally, represents the elapsed time from start to end of program execution on a running computer. It includes wait time and the execution time of other programs. Periods during which the computer is shut down are not counted. If computer utilization, which is the fraction of time the processor is executing the program, is constant, clock time will be proportional to execution time. To illustrate these three types of time, consider a transaction system serving a department store. In one week of calendar time, there may be 54 hours of clock time during which the system is running. There might be 25 hours of execution time for the transaction processing program itself. Note that when either execution time or clock time is used, any down time of a system is excluded.

There are four general ways of characterizing failure occurrences in time:

1. time of failure,
2. time interval between failures,

3. cumulative failures experienced up to a given time, and
4. failures experienced in a time interval.

These quantities are random variables, that is, their values are not known with certainty. There is a range of possible values, each associated with a probability of occurrence. For example, one does not really know when the next failure will occur. Only a set of possible times of failure occurrence and the probability of each is known. The probability of each time of occurrence is the fraction of cases for which it occurs. Note that “random” does not carry any connotation of true irrationality or unpredictability, as some mistakenly assume. “Random” means “unpredictable” only in the sense that the exact value is not known. However, the average value and some sense of the dispersion *are* known. “Random” does not mean “unaffected by other variables.” Although failure occurrence is random, it is decidedly affected by such factors as test strategy and the operational profile (see Section 1.3.1). Nor does “random” imply any specific probability distribution, which some erroneously assume as “uniform.”

There are at least two reasons for this randomness. First, the commission of errors by programmers, and hence the introduction of faults, is a very complex, unpredictable process. Therefore the locations of faults within the program are unknown. Second, the function to be executed by a program is generally unpredictable. For example, with a telephone switching system, how does one know what type of call will be made next?

A random process can be viewed as a set of random variables, each corresponding to a point in time. The process can be discrete or continuous, depending on whether time is discrete or continuous. The two principal characteristics of a random process are the form of the probability distributions of the random variables (for example, Poisson) and the form of the variation of the process with time.

Time variation can be viewed in terms of the *mean-value function* or the *failure intensity function*. The mean-value function represents the average cumulative failures associated with each time point. The failure intensity function is the rate of change of the mean value function or the number of failures per unit time. For example, one might say 1 failure per 1000 hours. Strictly speaking, the failure intensity is the derivative of the mean-value function with respect to time, and is an instantaneous value. A random process whose probability distribution varies with time is called *nonhomogeneous*. Most failure processes during test fit this situation.

Two principal factors affect failure behavior:

1. the number of faults in the software being executed, and
2. the operational profile.

The number of faults in the software is the difference between the number introduced and the number removed.

Faults are introduced when the code is being developed. Developers may introduce faults during original design or when they are adding new features, making design changes, or repairing faults that have been identified. The term *developed code*, defined as instructions that have been created or modified, is used deliberately. In general, only code that is new or modified results in fault introduction. Code that is *reused* without change from another application does not usually introduce any appreciable number of faults, except possibly in the code interfaces. It usually has been thoroughly debugged in the previous application. Note that fault removal introduces some new faults because it involves modification or writing of new code. Hopefully (if the programming staff is reasonably competent!), the new faults entered are fewer than the faults being removed.

Fault removal obviously cannot occur unless some means of detecting the fault in the first place exists. Fault removal resulting from execution depends on the occurrence of the associated failure. Occurrence depends both on the amount of execution time and on the operational profile. Faults can also be found without execution through inspection, compiler diagnostics, design or code reviews, or code reading.

Faults removed per failure can vary. For example, a failure correction team may remove 95 faults for every 100 failures reported. The efficiency depends on such factors as how well the circumstances surrounding the failure are documented and the degree of structuring and clarity of the program.

Finally, the failure process and hence software reliability is directly dependent on the operational profile for the program. For example, an electronic switching system being used in a business district may have a software reliability different from that in a residential area.

1.3.1 Operational Profile

Several secondary concepts are needed to define the operational profile. The execution of a program is commonly considered as being divided into runs, each of which is generally associated with some function that the program performs. Runs that are identical repetitions of each other are said to form a *run type*. The proportion of runs of various types may vary, depending on the functional environment. Examples of a run type might be

1. a particular transaction in a stock brokerage system,
2. a specific cycle in a closed-loop control system (for example, in a flight control system), or
3. an operation performed by an editor for a user.

During testing, the term “test case” is sometimes used instead of “run type.”

The second concept needed is that of an *input variable*. This is a variable that exists external to the program and is used by it. For a stock brokerage system, “security name” might be an input variable. There is usually a large quantity of input variables associated with the program. Each set of values of these variables characterizes an *input state*. The input state identifies the particular run type that is being made. Therefore, runs can always be classified by their input states. Again, in the case of the stock brokerage system, the input state might be characterized by particular values of security name, buyer, seller, price, etc. The set of all possible input states is known as the *input space*, illustrated in Fig. 1.

Similarly, an *output variable* is a variable that exists external to a program and is set by it. An *output state* is a set of values of all output variables associated with a run of a program. In the stock brokerage system, an output state might include a transaction confirmation report and monthly customer statements, among other items. It can now be seen that a failure involves a departure of the output state from what it is expected to be.

We define the *operational profile* as the set of run types that the program can execute along with their occurrence probabilities. Figure 1 shows two of many possible input states, A and B, with their probabilities of occurrence. The part of the operational profile for just those two states is shown in Fig. 2. A realistic operational profile is illustrated in Fig. 3. Note that the input states have been arranged on the horizontal axis in order of the probabilities of their occurrence without loss of generality. With the large number of input states that generally occur in reality, the operational profile appears to be a continuous curve. The operational profile is an extremely useful concept. However, it may not always be practical to enumerate all the input states and

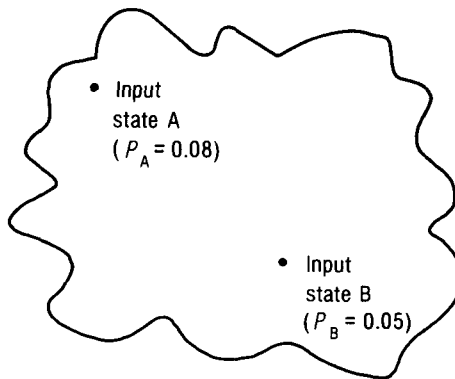


FIG. 1. Input space.

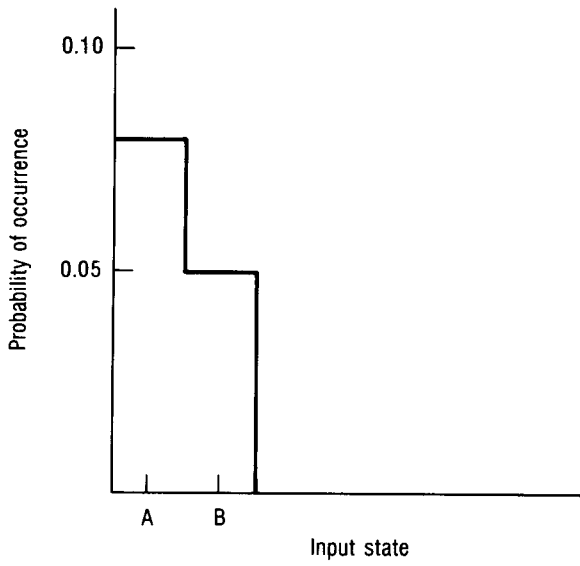


FIG. 2. Portion of operational profile.

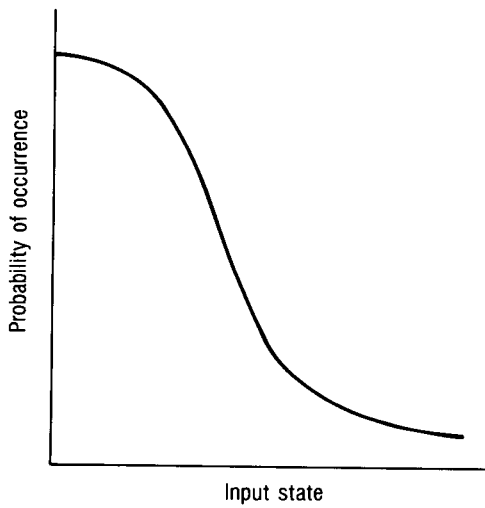


FIG. 3. Operational profile.

their probabilities in full detail, because of the large number that can be involved. This number can be reduced by the use of equivalence partitioning. Equivalence partitioning (Cox, 1981; Myers, 1979) groups input states that are very similar to one another and hence likely to have or not have a fault in common. One input state from each partition is selected with a probability equal to the total probability of occurrence of all states in the group. This drastically reduces the number of input states for selection and speeds up testing. The risk is that the ignored states may have independent faults which are then missed. With a reasonably small number of such classes, an operational profile based on the classes may be determined in practice. One source of data for such an operational profile is a survey of expected customer uses of the system.

1.3.2 *Software Reliability*

Software reliability is the probability of failure-free operation of a computer program for a specified period in a specified environment (operational profile). For example, a time-sharing system may have a reliability of 0.92 for 8 CPU hours for the “average user.” This system, when executed for 8 CPU hours, would operate without failure for 92 of these periods out of 100. As a result of the general way in which a “failure” is defined, the concept of software reliability can incorporate the notion of performance being satisfactory. For example, excessive response time at a given load level may be considered unsatisfactory, so that a routine must be recoded in more efficient form.

In order to have accurate reliability measurements during testing, runs should be selected randomly with the same probability as is expected to occur in operation. (Compensation for the use of different probabilities may be possible in some cases. See Musa *et al.* (1987).) The effect of selecting runs by plan rather than randomly depends on whether the instructions and machine states exercised are localized or not. If a localized part of the program is exercised, the failures that occur will be the result of faults in only that part. Fault removal will be more rapid than usual. Since the failure intensity curve will reflect just the part of the program examined, it will initially decrease more rapidly than would be expected. When a transition to a new set of instructions and machine states occurs, the failure intensity curve will probably rise due to the greater number of faults remaining in the new part of the program.

The definition implies that the input space must be “well covered” for accurate reliability measurement. To measure input space coverage, add the probabilities in the operational phase associated with the test runs selected. The coverage at first can be small when the input space is very large. The sum gives the probability that a run occurring during operation will have already been tested. Assume equal run lengths. Then any error in reliability

measurement due to insufficient coverage cannot exceed the difference between 1 and the coverage. Thus, input space coverage should be substantially closer to 1 than the level of reliability being measured. For example, one might attempt coverage of 0.9999 for a reliability measurement expected to be near 0.99. The probability that a run occurring in operation will not have been tested is 0.0001 and hence the reliability measurement will have an error of 0.0001 or less. This is about 1%.

Failure intensity is an alternative way of expressing reliability. In the preceding example the reliability of a particular system being 0.92 for 8 CPU hours is equivalently stated as the failure intensity being 0.01 failure per CPU hour. (Let R be the reliability, λ the failure intensity, and τ the execution time. Then $R(\tau) = \exp(-\lambda\tau)$.) Each alternative has its advantages. The failure intensity statement is more economical, since only one number is needed. However, the reliability statement is more convenient for taking the combination of reliabilities of components to get system reliability. If the risk of failure at any time, such as for a nuclear power plant, is of paramount concern, failure intensity may be the more appropriate measure. When proper operation of a system to accomplish some function with a time duration, such as a space flight, is required, reliability is often best. Figure 4 shows how failure intensity and reliability typically vary during a test period, as faults are removed. Note that failure intensity, like reliability, is defined with respect to a specified operational profile. The relationship between failure intensity and

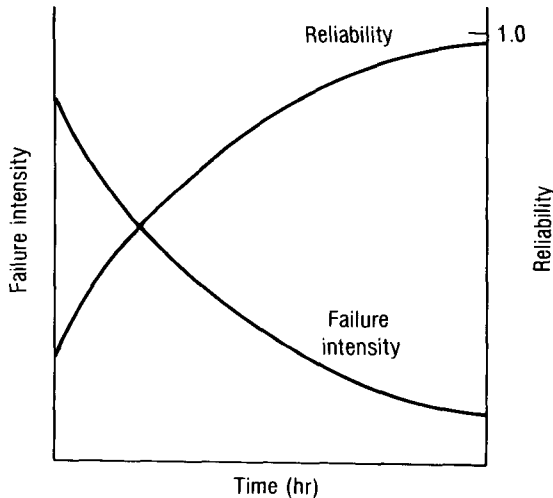


FIG. 4. Reliability and failure intensity.

reliability depends on the reliability model employed if these values are changing.

As faults are removed, failure intensity tends to drop and reliability tends to increase. When faults are being introduced, they often are included in new features or design changes that are all made at one time, resulting in a step increase in failure intensity and a step decrease in reliability. It is also common to introduce field repairs in batches. There can be a step increase or decrease in failure intensity, depending on whether new code and its faults or fixes predominate. There will be a corresponding decrease or increase in reliability. If a system is stable (the code is unchanging), both failure intensity and reliability will tend to be constant. This condition is common for a program that has been released to the field, with no changes in code and no fixes being introduced.

1.4 Relation to Hardware Reliability

Since the field of hardware reliability has been established for some time, one might ask how software reliability relates to it. The division between hardware and software reliability is actually somewhat artificial. Both may be defined in the same way, so that hardware and software component reliabilities can be combined to get system reliability. Both depend on the environment. The source of failures in software is design faults, while the principal source of failures that have been considered in hardware has generally been physical deterioration. But the concepts and theories developed for software reliability could really be applied to any design activity, including hardware design. Once a software (design) defect is properly fixed, it is in general fixed forever. Failure usually occurs only when a program (design) is exposed to an environment that it was not developed or tested for. Although manufacturing processes can affect the quality of physical components, the replication process for software (design) is trivial and can be performed to very high standards of quality. Since introduction and removal of design faults occur during software development, software reliability may be expected to vary during this period.

The "design reliability" concept historically has not been applied to hardware to any extent, because the incidence of failure due to wear and other physical causes has generally been considerably greater than that due to an unrecognized design problem. It was possible to keep hardware design failures low because hardware was generally less complex logically than software. Hardware design failures had to be kept low because retrofitting of manufactured items in the field was very expensive. The emphasis in hardware reliability may be starting to change now, however. There is greater awareness of the work that is going on in software reliability. Also, there is a growing

realization of the importance of design faults. The growing awareness is strengthened by the parallels that practitioners are drawing between software engineering and chip design.

A final characteristic of software reliability is that it usually changes continually during test periods. This happens either as new problems are injected when new code is written, or when repair action removes problems that exist in the code. Hardware reliability may change during certain periods, such as initial burn-in or the end of useful life. However, it has a much greater tendency toward a constant value than software.

The term *mean time to failure*, the average value of the next failure interval, is used in the hardware reliability field and to a decreasing extent for software reliability. The use of mean time to failure is attractive, because "larger" indicates "better." However, there are many cases in software reliability in which the mean time to failure is undefined. Failure intensity is preferred because it always exists. Further, failure intensities are simple to work with because they combine additively. Failure intensity and mean time to failure are inverses of each other, in an approximate nonrigorous sense. The term *mean time between failures* is used in the hardware reliability field when repair or replacement is occurring. It is the sum of mean time to failure and *mean time to repair*.

Despite the foregoing differences, software reliability theory has generally been developed in a way that is compatible with hardware reliability theory, so that system reliability figures may be computed using standard hardware combinatorial techniques (Shooman, 1986, Lloyd and Lipow, 1977).

2. Modeling Software Reliability

The main factors that affect software reliability are fault introduction, fault removal, and the operational profile. Fault introduction depends primarily on the characteristics of the developed code (code written or modified for the program) and the development process. The code characteristic with the greatest effect is size. Development process characteristics include the software engineering technologies and tools employed and the average level of experience of programmers. Note that code is developed when adding features or removing faults. Fault removal is affected by time, the operational profile, and the quality of the repair activity. Since most of the foregoing factors are probabilistic in nature and operate over time, software reliability models are generally developed as models of random processes. The models are distinguished from each other by the probability distribution of failure times or number of failures experienced and by the form of the variation of the random process with time. They express the general form of the relationship

between failures and factors mentioned. The possibilities for different general mathematical forms to describe the failure process are almost limitless. The specific form is determined by establishing the values of the parameters of the model through either (Hecht, 1977):

1. estimation—applying statistical inference procedures to the failure data, or
2. prediction—relating parameter values to characteristics of the software product and the development process (which can be done before failure data is available).

There will be uncertainty in the determination of the specific form. This is generally expressed in terms of *confidence intervals* for the parameters. A confidence interval represents a range of values within which a parameter is expected to lie with a certain confidence. For example, the 75% confidence interval of initial failure intensity may be 9 to 11 failures per hour.

After the specific form has been established, many different properties of the failure process can be determined. For many models there are analytic expressions for

1. the average number of failures experienced at any point in time,
2. the average number of failures in a time interval,
3. the failure intensity at any point in time, and
4. the probability distribution of failure intervals.

A good software reliability model has several important characteristics. It

1. gives good predictions of future failure behavior,
2. computes useful quantities,
3. is simple,
4. is widely applicable, and
5. is based on sound assumptions.

The term *prediction* has two different but commonly used meanings in software reliability, “prediction of parameter values” and “prediction of failure behavior.” One must be careful to distinguish them. Prediction of future failure behavior is based on the assumption that the values of model parameters will not change for the period of prediction. If the net effect of the opposing influences of fault introduction and fault removal should change substantially, one must either compensate for the change or re-estimate the model parameters from the failure data. Incorporating such changes into the models themselves is usually impractical because of the added complexity.

Although it may not be stated explicitly, most software reliability models are based on a stable program and a constant operational environment. If the program or operational profile change, they often do so and are usually handled in a piecewise fashion. Thus, the models focus mainly on fault removal. Most models can account for the effects of slow fault introduction, as long as the average net long-term effect is to be a decrease in failure intensity. If neither fault introduction, fault removal, nor operational profile changes are occurring, the failure intensity will be constant and the model should simplify to accommodate this. It is possible to compensate for programs or operational profiles that are changing (see Musa *et al.*, 1987).

For a program in the field, it is common to defer installation of both new features and fault fixes to the next release. If the operational profile is constant, the program will have a failure intensity that is constant for any given release and changes at the introduction of the next release.

A good model enhances communication on a project and provides a common framework for understanding the software development process. It also enhances visibility to management, the customer, and others. These characteristics are valuable, even if the projections made with the model are not particularly accurate in a specific case.

The usefulness of a software reliability model depends on the development of tools and the accumulation of a body of lore from practical experience. This effort generally requires several person years. Hence a project needs to select a model that is well established in practice, so that its implementation requires a very small fraction of project resources.

Sometimes it has been suggested that a range of models be applied to each project. The ones that perform best (or some weighted combination of them) would be used. This approach may be suitable for research investigations, but the authors consider the use of more than one or two models conceptually and economically impractical for real projects.

2.1 General Model Characteristics

A *software reliability model*, as previously noted, is generally expressed as a random process that describes the behavior of failures with time. Characterization of the model in time is usually accomplished by specifying either the mean-value function or the failure intensity function. The parameters are dependent on repair activity and program change and properties of the software product and the development process. Program change is frequently handled separately by applying models in a piecewise fashion or by adjusting failure times to what they would have been if the entire program had been there. The adjustment is based in part on the inherent faults introduced by the developed code that is added. Size, complexity, and structure are the

significant characteristics of the product. Properties of the development process include the software engineering technologies and tools used and the average level of experience of personnel. The origin of time may be arbitrarily set, but it is frequently the start of system testing.

Software reliability models almost always assume that failures are independent of each other. They may do this by assuming that failure times are independent of each other or by making the Poisson process assumption of independent increments. In most cases failures are independent since they are the result of two processes: the introduction of faults and their activation through the selection of input states. Both of these processes are random, and hence the chance that one failure would affect another is small. The independence conclusion is supported by a study of correlograms of failure data from 15 projects (Musa 1979c), which found no significant correlation.

Independence is sometimes disputed because the "two process" situation is not well known. Some argue that programmers tend to have patterns of errors (that is, one error may influence the next one). Related faults may indeed be produced but this does not imply the existence of related failures. Similarly, it is sometimes stated that one failure can prevent access to certain code and thus prevent another failure from occurring that is associated with that code. This prevention may occur occasionally in unit testing but it is much less common in practice in system testing or in the operational phase. In any case, it is even rarer to have the fault causing the first failure cause a second fault to be introduced into the "hidden" code.

We will concentrate on "in the large" models of software reliability, since they are generally the most useful type. These models deal with the major factors and effects that are the principal concerns of software engineers and managers.

Many published models employ calendar time or do not specify the type of time. Substantial evidence demonstrates the superiority of execution time over calendar time for initial modeling, however (Trachtenberg, 1985; Musa and Okumoto, 1984b; Hecht, 1981). Execution time is preferable because it best characterizes the failure-inducing stress placed on software. Consider two different facilities associated with an operating system, an editor and a Bessel function routine. The editor, perhaps executing two to three CPU hours daily, is certainly more stressed than the Bessel function routine, which might execute for only a few CPU minutes. Some models, originally developed as calendar-time models or without regard to the type of time, are being interpreted as execution-time models. In such cases, calendar-time components should be developed if possible. The calendar-time component converts execution time into calendar time, yielding another time scale that is needed by managers and engineers for several purposes.

2.1.1 *Random Process*

Both the human error process that introduces faults into code and the run selection process that determines which code will be executed are dependent on many time-varying variables. The use of a random process model is appropriate for such a situation.

Theoretically the selection of runs could be planned or manipulated during testing. The tester has at least partial control of the environment, so that one might think that random selection of input states would be a poor model. However, the introduction of faults into code and the relationship between input state and code executed are usually both sufficiently complex processes that one can not predict which input states are more likely to yield failures. Consequently, a deterministic selection of input states will not have a deterministic effect on reliability, so that a random process is still a good model of failure behavior. Of course, if the relative frequencies of selection have changed, then the operational profile has changed and that will affect the reliability.

2.1.2 *With and Without Fault Removal*

Software reliability models must cover programs whose faults are being removed when failures occur and those whose faults are not. These situations can occur in either the test or the operational phase, but “no fault removal” is usually associated with the latter. The situation of “no fault removal” is simple and is covered in this section. The situation of “with fault removal” is the subject of most of this article. The term “no fault removal” really means “deferred fault removal.” Failures are noted, but underlying faults are not removed from the program until the next release.

The failure intensity is constant for the duration of the release. Thus, the failure process can be modeled by a homogeneous Poisson process. This implies that the failure intervals are exponentially distributed, and that the number of failures in a given time period has a Poisson distribution. If the failure intensity is λ and the period of execution of the program is τ , then the number of failures during this period has a Poisson distribution with parameter $\lambda\tau$.

As mentioned before, the principal factor that makes reliability vary with time is the correction of faults that have caused failures. In general, the time of correction does not coincide with the time of failure. This could lead to appreciable complication in characterizing the failure process. However, we handle the problem by assuming instantaneous repair and not counting recurrences of the same failure. They would be counted, however, if the recurrences were due to inability to locate and fix the fault.

2.1.3 Estimation

We have noted that if failure data is available, then parameters may be determined through statistical estimation. A model and an inference procedure are commonly associated. Together, they provide projection through time. Without a model, it isn't possible to make inferences, because the size of the failure sample would be one. The model provides the structure that relates behavior at different times, effectively yielding a sample of reasonable size.

The inference procedure that has been historically associated with a model is not necessarily the "best." One may wish to consider alternatives. Inference also includes the determination of the range of uncertainty. Either confidence intervals for the parameters are established or, in the case of Bayesian inference, posterior probability distributions for significant quantities are determined. The determination of ranges of uncertainty is usually extended to quantities that are derived from the models as well.

Failure data is usually available in the form of times of failures or number of failures in a given time interval. The "time of failure" form is the most precise and contains the most information. Time can be specified in either calendar time, the actual chronological period that has passed, or execution time, the processor (CPU) time accumulated.

2.2 Selecting a Model

Many different software reliability models have been proposed, but software engineers and managers have received little guidance as to which models may be best for their application.

Early investigators (for example, Sukert (1979) and Schick and Wolverton (1978)) attempted to compare some of the different models but unfortunately were handicapped by insufficient data of high quality and an absence of standards for the comparison criteria to be employed. Since then more good-quality failure data on a variety of software systems has become available (for example, Musa (1979a)). Several researchers jointly proposed comparison criteria, incorporating comments from many reviewers in the field (Iannino *et al.*, 1984). To provide a sound and efficient basis for comparison, Musa and Okumoto (1983) classified the various models and explored the assumptions underlying software reliability theory.

Let's examine the criteria. Some of them are general ones that apply to all types of models, even though we discuss them in the specific context of software reliability. Only "intrinsic" criteria have been selected. Attributes such as documentation quality and human interface quality of supporting tools are important, but they depend on the implementation and are not characteristic of the model itself. Note that functional validity and "insight

gained” are valuable characteristics of models, even though not mentioned by Iannino *et al.* (1984).

These criteria can be used for assessment or for determining the absolute worth of a model as well as for comparison. Comparison should be done over a variety of software systems. Evaluation of the models will probably not be application-dependent, but one must watch for this possibility. It would not be sensible to compare models with poor-quality data, or, worse, data of unequal quality. However, as will be noted in Section 2.2.4, tolerance of or capability to compensate for certain types of poor data is a desirable quality in a model.

Comparisons will probably cause some models to be rejected because they meet few of the criteria discussed here. On the other hand, there may or may not be a clear choice between the more acceptable models. The relative importance to be placed on the different criteria may depend on the situation in which the model is being applied. When comparing two models, one must consider all criteria simultaneously and not eliminate models on one criterion before considering the others, unless predictive validity is grossly unsatisfactory. It is not expected that a model must satisfy *all* criteria to be useful.

The criteria include predictive validity, capability, quality of assumptions, applicability, and simplicity. Each is discussed in more detail in the following sections.

2.2.1 Predictive Validity

Predictive validity is the capability of the model to predict future failure behavior from present and past failure behavior (that is, data). The capability is most significant when failure behavior is changing. Hence, it is usually considered for a test phase, although it can be applied to the operation phase. Note that “predictive validity” does not carry the connotation of “prediction” in referring to determination of parameters from program characteristics.

There are at least two general ways of viewing predictive validity, based on the two equivalent approaches to characterizing the failure random process, namely

1. the number-of-failures approach, and
2. the failure-time approach.

Various detailed methods, some representing approximations to predictive validity, have been proposed. It has not been determined if one is superior at the present time.

The number-of-failures approach may yield a method that is more practical to use than the failure-time approach. In the former approach, the failure random process is described by $M(\tau)$, which represents failures experienced

by execution time τ . Such a process is characterized by specifying the distribution of $M(\tau)$, including the mean value function $\mu(\tau)$.

Assume that q failures have been observed by the end of test time τ_q . The failure data up to time $\tau_e (\leq \tau_q)$ is used to estimate the parameters of $\mu(\tau)$. If we substitute the estimates of the parameters in the mean-value function we obtain the estimate of the number of failures $\hat{\mu}(\tau_q)$ by τ_q . Compare the estimate with the actually observed number q . Repeat for various values of τ_e .

Check the predictive validity by plotting the relative error $(\hat{\mu}(\tau_q) - q)/q$ against the normalized test time τ_e/τ_q . The error approaches zero as τ_e approaches τ_q . If the error is positive, the model is overestimating; if the error is negative, the model is underestimating. Numbers closer to zero imply more accurate prediction and hence the better model.

The normalization used in this approach lets you overlay relative error curves obtained from different sets of failure data. For an overall conclusion about the relative predictive validity of models, one may compare plots of the medians of these error points (taken with respect to the various data sets). A model would be superior if it yields the curve closest to 0.

In the failure-time approach, you characterize the failure random process by failure times (execution or calendar). The cumulative distribution function provides the most general description of the random process. Other quantities, such as mean time to failure (if it exists), failure intensity, and reliability, can be obtained from it. Assume that m failure times are observed during testing, denoted by $\tau_1, \tau_2, \dots, \tau_m$. The predictor distribution for the i th failure time incorporates model and inference procedure and is written as $\tilde{F}_i(\tau_i | \tau_1, \dots, \tau_m)$.

Note that $i > m$, since we are concerned with prediction. Predictive validity is determined by comparing predictor distributions of failure times from the model and its associated inference procedure with values experienced. This is done based on the u -plot (Littlewood and Verrall, 1973 and Section 4.2.1).

The i th failure time can be predicted from the first m failure times. Let $k = i - m$. Then k signifies the prediction length or number of times we are predicting ahead. Since many data points are needed to check the predictive validity of $\tilde{F}_i$, either m can be fixed and k varied ($k > 0$) or k can be fixed and m varied (i varies in both cases). In the former case, we are making predictions of various lengths using a fixed sample size. In the latter case, we are making predictions of a fixed length based on different sample sizes. Predictive validity should be checked from both viewpoints. If one model has equal predictive validity with another but attains this capability earlier (that is, for a smaller failure sample), it may be superior.

As previously noted, most models have been developed in association with a particular inference procedure (such as maximum likelihood or least squares). However, other inference procedures should be considered if prediction is

poor. We can examine the quality of inference by generating simulated failure intervals for the model with assumed parameter values and using them in the prediction scheme (combination of model and inference procedure). This method removes any effects of the model. Consistent differences between the predictor distribution and the actual distribution will represent the effects of inference alone. Alternatively, the distributions of the parameter estimates can be compared with the observed values to evaluate inference quality. However, different estimation methods probably will not have a substantial effect on predictive validity (Musa *et al.*, 1987).

2.2.2 Capability

Capability is used to indicate the ability of the model to estimate with satisfactory accuracy quantities needed by software managers, engineers, and users to plan and manage software development projects or the use of software they are responsible for. The degree of capability is determined from relative importance of the quantities as well as their number. The quantities, in approximate order of importance, are

1. present failure intensity, mean time to failure, or reliability,
2. expected date of reaching a specified failure intensity, mean time to failure, or reliability objective, and
3. human and computer resource and cost requirements related to meeting the objective. It is assumed that the objective can be varied and that dates and resource requirements can be computed for many objectives, if desired. If a date cannot be computed, then the achievement of the objective should be described in terms of additional execution time or failures experienced.

Any capability of a model for prediction of software reliability in the system design and early development phases is especially valuable because of its importance for system engineering and planning purposes. Predictions are made through measurable characteristics of the software (size, complexity, structure, etc.), the software development environment, and the operational profile.

2.2.3 Quality of Assumptions

If it is possible to test an assumption, the degree to which it is supported by data is an important consideration, particularly if the assumption is common to an entire class of models. If not, its *plausibility* should be evaluated from the viewpoint of logical consistency and software engineering experience.

Does it agree with other available information about software and software development? Finally, is the assumption clear and explicit, so that one can determine whether a model applies to particular circumstances?

2.2.4 *Applicability*

Another important characteristic of a model is its applicability. A model should be judged on its degree of applicability across software systems of varying size, structure, function, etc. It should be usable in different development environments, different operational environments, and different life-cycle phases. Nevertheless, if a particular model gives outstanding results for only a narrow range of systems or development environments, it should not necessarily be eliminated.

A model should be capable of dealing with special situations commonly encountered in practice. It can either handle them directly or be compatible with procedures that can deal with them. The most common situations are

1. program evolution,
2. classification of failures into different categories by severity,
3. incomplete failure data or data with measurement uncertainties (of course, some loss of predictive validity may be expected), and
4. operation of the same program on computers of different speeds.

Finally, a model should be robust with respect to departures from its assumptions, errors in the data or parameters it employs, and unusual conditions.

2.2.5 *Simplicity*

A model should be simple in three respects. Most important is that data collection for determining parameters should be simple and inexpensive; otherwise, the model will not be used. Second, the model needs to be simple in concept. Software engineers without extensive mathematical background should be able to understand the model and its assumptions, so that they can determine when it is applicable and the extent to which it may diverge from reality in an application. Parameters should have physical interpretations, so that software engineers can estimate the values of the parameters when data is not available. The number of parameters in the model is also an important consideration for simplicity. Incidentally, one must compare the number of parameters on a common basis (for example, one should not include calendar-time component parameters for one model and not another).

Finally, a model must be readily implementable as a program that is a practical management and engineering tool. This means that the program must run rapidly and inexpensively without excessive manual intervention.

Although simplicity is just common sense, it is easy to get carried away with intellectual elegance and power. This appeal must be consciously resisted in modeling, whose purpose, after all, is pragmatic.

2.3 Execution-Time Modeling

This section presents a general approach to the development of a large group of software reliability models that was undertaken to help

1. unify the field,
2. highlight relationships among models,
3. suggest possible new models, and
4. make comparisons among the models.

The approach is based on Markov processes. These processes are helpful in modeling random behavior of software in time, such as faults remaining at and failures experienced by time τ . Markov processes are further characterized by the distribution of the number of failures over time. The two most important distributions are the Poisson and binomial (see Fig. 5). Models based on the binomial distribution are finite failures models, that is, they postulate that a finite number of failures will be experienced in infinite time. Models based on the Poisson distribution can be either finite-failures or infinite-failures models depending on how they are specified. Finite-failures Poisson-type models can be shown to be a generalization of binomial-type models.

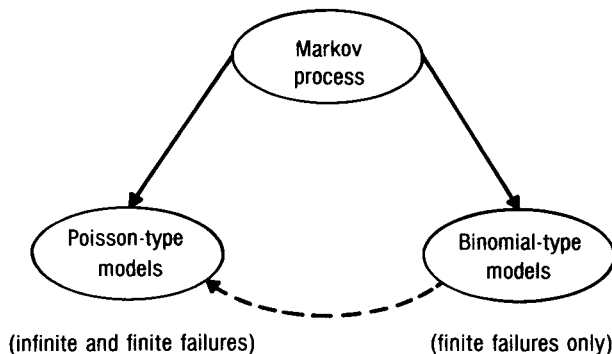


FIG. 5. Characterization of execution-time models.

TABLE I
MARKOV SOFTWARE RELIABILITY MODELS

Poisson-type	Binomial-type
Crow (1974)	Jelinski and Moranda (1972)
Musa (1975)	Shooman (1972)
Moranda (1975)	Schick and Wolverton (1973)
Schneidewind (1975)	Wagoner (1973)
Goel and Okumoto (1979b)	Schick and Wolverton (1978)
Yamada <i>et al.</i> (1983)	Littlewood (1981)

Table I shows how some of the published models can be classified under this approach. The geometric de-eutrophication model of Moranda (1975) and the Bayesian model of Littlewood and Verrall (1973) are among the few published models that are not Markov. Miller (1986) examined characterizations, properties, and examples of various models using exponential-order statistical models, which are a subset of Markov models.

A Markov process has the property that the future of the process depends only on the present state and is independent of its history. The assumption may be reasonable for a software failure process, which is principally dependent on faults remaining and the operational profile (the latter is not dependent on the past). Markov processes can be characterized by the amount of time spent in a state and the transitions between states.

To illustrate, consider a computer program that has total faults u_0 at the start of system testing. A fault causes a software failure when the program is executed for certain input states. Failures occur randomly in time because of the random selection of input states and uncertainty of fault location. As each failure occurs, an attempt is made to identify and remove the fault that caused the failure. However, the fault may not be removed with certainty. Furthermore, the repair activity may introduce a new fault.

In this example, the number of faults remaining at a future time is not known with certainty. Such random behavior can be formulated as a Markov process. Let $U(\tau)$ denote a random variable indicating faults remaining at time τ . Assume that there are u_0 faults at time $\tau = 0$, that is, $U(0) = u_0$ with probability one. Figure 6 shows a possible realization of the $U(\tau)$ process, where τ_i represents the i th failure time. Note that one fault is removed at τ_1 and another at τ_3 . The repair activity is imperfect at τ_2 (that is, no fault is removed) and a new fault is introduced at τ_4 because of the repair process.

The Markov property for the $U(\tau)$ process is stated as follows: For an interval $\Delta\tau$, faults remaining at time $\tau + \Delta\tau$ depend only on the present state at

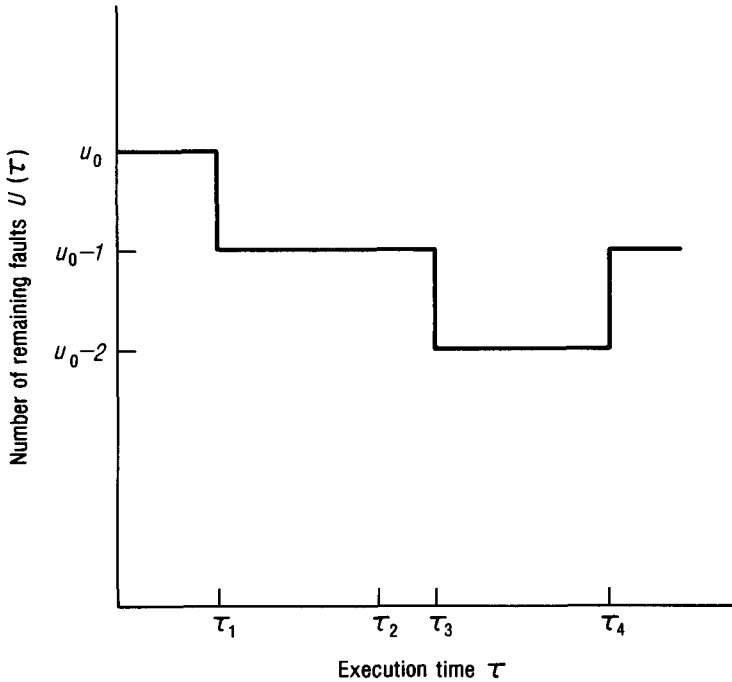


FIG. 6. A possible realization of the number of faults $U(\tau)$ remaining at time τ .

time τ . In other words, the conditional probability $P[U(\tau + \Delta\tau) = j \mid U(\tau) = i]$ describes the future behavior of the $U(\tau)$ process. The conditional probability or the transition function of the process $P_{ij}(\tau, \Delta\tau)$ is usually dependent on both τ and $\Delta\tau$ (in addition to i and j). The probability that $U(\tau + \Delta\tau)$ is equal to j is given by

$$P[U(\tau + \Delta\tau) = j] = \sum_i P_{ij}(\tau, \Delta\tau) P[U(\tau) = i]. \quad (2.1)$$

Goel and Okumoto (1978, 1979b) and Shanthikumar (1981) studied the $U(\tau)$ process with imperfect debugging. Kremer (1983) further incorporated the possibility of introducing a new fault during the repair action. His approach provides useful insight. Unfortunately, expressions for reliability and other quantities of interest are complex and it is difficult to apply this approach in practice.

Another, more practical way of looking at the problem is to formulate failures experienced by time τ as a Markov process. One can actually observe failures experienced, whereas faults cannot be clearly observed in reality.

Therefore, the former approach is taken. In the following section, a nonhomogeneous Poisson process is developed as a special case of a Markov process to study the random behavior of failures experienced in a computer program. Existing Poisson-type models of both finite and infinite-failures category are described as special cases of the Poisson process. In Section 2.3.2 binomial-type models are described as special cases of a Markov process. It is then shown in Section 2.3.3 how a generalization of binomial-type models leads to Poisson-type finite-failures models. The latter are, of course, special cases of general Poisson-type models. Poisson-type models of the finite-failures category and binomial-type models are compared in Section 2.3.4. A fault reduction factor for Poisson-type models is discussed in Section 2.3.5.

2.3.1 General Poisson Models

In this section we will represent the software failure process using a nonhomogeneous Poisson process with failure intensity $\lambda(\tau)$. Existing Poisson-type models (of both finite and infinite-failures category) are included as special cases of the underlying general nonhomogeneous Poisson process. First we will present basic assumptions and characteristics.

Let $M(\tau)$ denote failures experienced by time τ . Figure 7 shows a possible realization of the $M(\tau)$ process, where τ_i represents the i th failure time. Note that the $M(\tau)$ process increases in value by 1 whenever a failure occurs. The $M(\tau)$ process for the Poisson-type models is derived based on the following assumptions:

1. There are no failures experienced at time $\tau = 0$, that is, $M(0) = 0$ with probability 1.
2. The process has independent increments. In other words, the number of failures experienced during $(\tau, \tau + \Delta\tau]$, that is, $M(\tau + \Delta\tau) - M(\tau)$, is independent of the history. This assumption implies the Markov property that the future $M(\tau + \Delta\tau)$ of the process depends only on the present state $M(\tau)$ and is independent of its past $M(x)$ for $x < \tau$.
3. The probability that a failure occurs during $(\tau, \tau + \Delta\tau]$ is $\lambda(\tau)\Delta\tau + o(\Delta\tau)$, where $\lambda(\tau)$ is the failure intensity of the process and the function $o(\Delta\tau)$ satisfies the condition

$$\lim_{\Delta\tau \rightarrow 0} \frac{o(\Delta\tau)}{\Delta\tau} = 0.$$

This means that the second- or higher-order effects of $\Delta\tau$ are negligible.

4. The probability that more than one failure occurs during $(\tau, \tau + \Delta\tau]$ is $o(\Delta\tau)$.

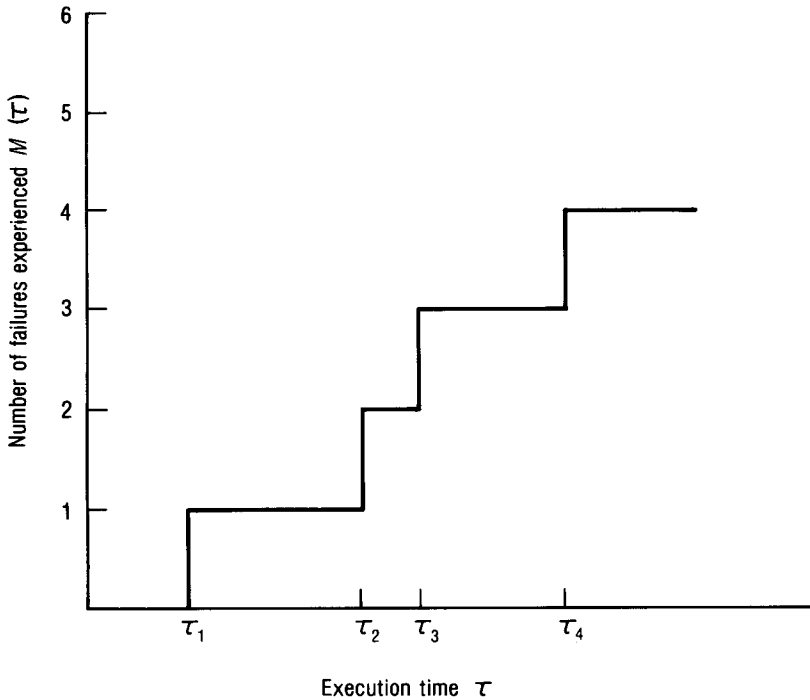


FIG. 7. A possible realization of the number of failures $M(\tau)$ experienced by time τ .

Many relationships can be derived based on these assumptions. The derivations can be found in standard books on stochastic processes (for example, Cinlar (1975)) and are not repeated here.

Table II provides a summary of the relationships that have been developed for a general Poisson model. The table can be used for obtaining relationships for a particular Poisson-type model. Note that the $M(\tau)$ process has a mean and variance of $\mu(\tau)$, which is called the mean-value function of the process. The failure intensity and the mean value function are related by

$$\mu(\tau) = \int_0^{\tau} \lambda(x) dx. \quad (2.2)$$

Therefore, the $M(\tau)$ process can be particularized by specifying the mean-value function or the failure intensity function.

2.3.2 Binomial-Type Models

Binomial-type models are developed based on the following assumptions:

1. Whenever a software failure occurs, the fault that caused it is removed instantaneously.
2. There are u_0 faults in the program.
3. Each failure, caused by a fault, occurs independently and randomly in time, according to the per-fault hazard rate $z_a(\tau)$. In other words, the hazard rates for all faults are the same.

Assumption 1 implies that the binomial-type models do not permit consideration of faults that cannot be located (that is, debugging is perfect), extra faults found through code inspection, or faults spawned during correction of

TABLE II
DERIVED RELATIONSHIPS FOR GENERAL POISSON AND BINOMIAL MODELS

Random quantities	General Poisson model	General binomial model
Failures experienced		
$P[M(\tau) = m]$	$\frac{[\mu(\tau)]^m}{m!} \exp[-\mu(\tau)]$	$\binom{u_0}{m} [F_a(\tau)]^m [R_a(\tau)]^{u_0-m}$
$E[M(\tau)]$	$\mu(\tau)$	$u_0 F_a(\tau)$
$\text{var}[M(\tau)]$	$\mu(\tau)$	$u_0 F_a(\tau) R_a(\tau)$
Failures remaining		
$P[Q(\tau) = q]$	$\frac{[\mu(\infty) - \mu(\tau)]^q}{q!} \exp\{-[\mu(\infty) - \mu(\tau)]\}$	$\binom{u_0}{q} [F_a(\tau)]^{u_0-q} [R_a(\tau)]^q$
$E[Q(\tau)]$	$\mu(\infty) - \mu(\tau)$	$u_0 R_a(\tau)$
$\text{var}[Q(\tau)]$	$\mu(\infty) - \mu(\tau)$	$u_0 R_a(\tau) F_a(\tau)$
Failure time		
$P[\tau_i \leq \tau]$	$\sum_{j=1}^{\infty} \frac{[\mu(\tau)]^j}{j!} \exp[-\mu(\tau)]$	$\sum_{j=1}^{u_0} \binom{u_0}{j} [F_a(\tau)]^j [R_a(\tau)]^{u_0-j}$
Program hazard rate		
$z(\tau'_i \tau_{i-1})$	$\lambda(\tau_{i-1} + \tau'_i)$	$(u_0 - i + 1) z_a(\tau_{i-1} + \tau'_i)$
Reliability		
$R(\tau'_i \tau_{i-1})$	$\exp\{-[\mu(\tau_{i-1} + \tau'_i) - \mu(\tau_{i-1})]\}$	$\exp\left[-(u_0 - i + 1) \int_{\tau_{i-1}}^{\tau_{i-1} + \tau'_i} z_a(\tau) d\tau\right]$
Failure Intensity		
$\lambda(\tau)$	$\frac{d\mu(\tau)}{d\tau}$	$u_0 f_a(\tau)$

previous faults. Hence, for binomial-type models the net number $N(\tau)$ of faults corrected by time τ is the same as the number $M(\tau)$ of failures experienced by τ . The total failures experienced in infinite time, v_0 , will be equal to the inherent faults u_0 . Denoting by $Q(\tau)$ a random variable representing failures remaining at time τ , yields

$$Q(\tau) = u_0 - M(\tau). \quad (2.3)$$

Note that u_0 could vary with changes in the operational profile, the operational history of the program, and the effectiveness of repair actions. But binomial-type models assume that the operational profile is fixed, and that from a macroscopic viewpoint there is an "average" history and average effectiveness of repair actions. Hence the number of failures that would occur in infinite time can be viewed as a fixed constant.

Each failure is not really totally independent of each other failure. One failure could influence succeeding failures by its individual impact on the effectiveness of repair. For example, failure A might conceivably reveal the possibility of failure C occurring to a greater extent than failure B would. This could result in repairs that more greatly reduce the probability that failure C would occur. Also, a failure may prevent another failure from occurring because it may prevent a particular code path from being executed. The latter situation occurs more frequently in unit testing than in system testing. Overall, dependence effects are, on the average, secondary, and can be neglected.

Table II also provides a summary of the relationships that have been developed for a general binomial model. The table is useful for obtaining relationships for a particular binomial-type model.

2.3.3 Poisson-Type Models (Finite Failures)

Finite-failures Poisson-type models can be developed directly from the binomial-type models. The development is necessary to relate the per-fault cumulative failure probability to the mean-value function. Then expressions for various quantities of interest can be readily found from those developed for the general Poisson model discussed in Section 2.3.1.

Assumption 3 made for the binomial-type models is also applied here. Recall from Assumption 2 that the binomial-type models assume a fixed number of faults remaining at $\tau = 0$, that is, $U(0) = u_0$ with probability 1. This assumption is modified for developing Poisson-type models as follows:

- 2a. The total number of faults remaining in the program at $\tau = 0$ is a Poisson random variable with a mean of ω_0 .

Since total faults is an extraordinarily complex function of many factors (for example, program size, complexity, and human performance), it is appropriate

to consider it as a random variable. The Poisson distribution is a reasonable assumption, since the number of faults is a count of essentially independent events. Note that Langberg and Singpurwalla (1985) took this approach to derive an exponential-class Poisson-type model from a binomial-type model from a Bayesian point of view.

The distribution of $M(\tau)$ is obtained as

$$P[M(\tau) = m] = \sum_{x=0}^{\infty} P[M(\tau) = m | U(0) = x] P[U(0) = x], \quad (2.4)$$

which may be viewed as a mixture distribution. The conditional probability of $M(\tau) = m$ given $U(0) = x$ is zero if $x < m$. For $x \geq m$ it is given by (see Table II with $u_0 = x$)

$$\binom{x}{m} [F_a(\tau)]^m [1 - F_a(\tau)]^{x-m}. \quad (2.5)$$

Therefore Equation (2.4) becomes

$$\begin{aligned} P[M(\tau) = m] &= \sum_{x=0}^{\infty} \binom{x}{m} [F_a(\tau)]^m [1 - F_a(\tau)]^{x-m} \frac{\omega_0^x}{x!} \exp(-\omega_0) \\ &= \frac{[\omega_0 F_a(\tau)]^m}{m!} \exp(-\omega_0) \sum_{x=m}^{\infty} \frac{\{\omega_0 [1 - F_a(\tau)]\}^{x-m}}{(x-m)!} \\ &= \frac{[\omega_0 F_a(\tau)]^m}{m!} \exp[-\omega_0 F_a(\tau)]. \end{aligned} \quad (2.6)$$

This is a Poisson distribution with a mean (and equivalent variance) of

$$\mu(\tau) = \omega_0 F_a(\tau). \quad (2.7)$$

Thus the failure process $M(\tau)$ is Poisson. Differentiating Equation (2.7) yields the failure intensity as

$$\lambda(\tau) = \omega_0 f_a(\tau). \quad (2.8)$$

As can be seen in the analysis, the principal difference between binomial-type and Poisson-type models is the way that the number of remaining faults is treated (that is, a fixed number for the binomial-type and a Poisson random variable for the Poisson-type).

All quantities of interest for finite-failures Poisson-type models can be found by substituting Equation (2.7) or Equation (2.8) into the appropriate equations in Table II.

2.3.4 Comparison of Binomial-Type and Poisson-Type Models

The characteristics of finite-failures Poisson-type models and binomial-type models are briefly compared here. The basic assumptions have been summarized in Assumptions 1, 2 and 3 for the binomial-type models and in Assumptions 1, 2a and 3 for the Poisson-type models, respectively.

First, both binomial-type and Poisson-type models assume that the hazard rates are the same for all faults, as shown by Assumption 3.

Assumptions 2 and 2a indicate that the Poisson-type models incorporate the case when the inherent number of faults is not known with certainty. There is not a fixed number of inherent faults as with the binomial-type models, but the expected value ω_0 of inherent faults is known. The parameter ω_0 in the Poisson-type model is, consequently, not restricted to an integral number of faults. The number of faults in a program is dependent on many presently unknown factors besides the known factors of program characteristics (for example, size, and complexity) and the development environment. Hence, it is reasonable to model initial faults as a random variable.

Poisson-type models also accommodate, in an approximate fashion, imperfect debugging and the introduction of new faults as the result of repair action. For example, in the case of imperfect debugging discussed previously, a Poisson model may be used with the mean-value function given by Equation (2.7).

The principal relationships for both binomial-type and Poisson-type models are summarized in Table II, so that a comparison can easily be made. For example, one can compare the characteristics of the program hazard rates. The binomial-type models have discontinuous program hazard rates with the discontinuities at the failures. The Poisson-type models do not have discontinuities in the program hazard rate. Binomial-type models are more realistic in that repairs *do* cause discontinuities in the failure intensity. However, the repairs occur at some variable (perhaps reasonably modeled as "random") time *after* the failures. This fact favors Poisson-type models.

The parameter estimation procedure for the Poisson-type models is the same as for the binomial-type (Musa *et al.*, 1987). This suggests that binomial-type models can be closely approximated by the Poisson-type.

2.3.5 Fault Reduction Factor for Poisson-Type Models

Poisson-type models also permit consideration of faults that cannot be located, extra faults found through code-inspection, and spawned faults. The fault removal process is characterized on an average basis by assuming that the fault correction rate is proportional to the hazard rate. Musa (1975) called this proportionality constant a fault reduction factor, denoted by B . The

“average basis” is reasonable because the useful applications of software reliability theory are primarily macroscopic in nature.

For a given number of faults ω_0 , the expected number of failures will be $\nu_0 = \omega_0/B$. Denote the cumulative distribution function of time to remove a fault by $G_a(\tau)$. The exact expression for the distribution of $M(\tau)$ is difficult to obtain, but it may be approximated by the Poisson distribution,

$$P[M(\tau) = m] = \frac{[\nu_0 G_a(\tau)]^m}{m!} \exp[-\nu_0 G_a(\tau)]. \quad (2.9)$$

The mean-value function is given by

$$\mu(\tau) = \nu_0 G_a(\tau), \quad (2.10)$$

and the failure intensity function is

$$\lambda(\tau) = \nu_0 g_a(\tau), \quad (2.11)$$

where $g_a(\tau)$ is the probability density function associated with $G_a(\tau)$.

If the actual repair process occurs at the time of failure, the Poisson-type model does not fit as well as the binomial-type. However, the effectiveness of the repair action is often imperfect, and the degree of imperfection varies randomly from repair to repair. The Poisson-type model offers the promise of approximating reality more closely because of its capability for handling imperfect repair.

2.4 Calendar-Time Modeling

So far the focus has been on the theory of the execution-time modeling of software reliability. This type of modeling allows the characterization and prediction of failure intensity as a function of execution time. Unfortunately, projections of when failure intensity objectives will be met in terms of execution time are not too useful when talking with managers and engineers. For example, stating that a system will attain its reliability goal in 21.7 hours of CPU time is not helpful. Most managers and engineers want to know the date on which the objective will be reached. This was the motivation for developing a second component of software reliability modeling, the calendar-time component, which is used to characterize and predict the passage of calendar time with execution time. With it, projections of when failure intensity objectives will be met can be made in terms of calendar dates rather than execution time. Software reliability models with an execution-time component *and* calendar-time component are the most desirable and useful to managers. Calendar-time modeling is described here in some detail.

2.4.1 Concept of Limiting Resources

The resources that govern the pace of testing for almost all software projects are

1. failure-correction personnel,
2. failure-identification personnel, and
3. computer time.

During system testing of a project of reasonable size, testing and failure correction are usually performed by different people. Hence, we will consider failure-identification and failure-correction personnel as separate resources. If resources additional to those listed above are employed, a generalization of the derivation that follows is straightforward. Figure 8 illustrates the usual interaction of these resources during the system test phase of a project.

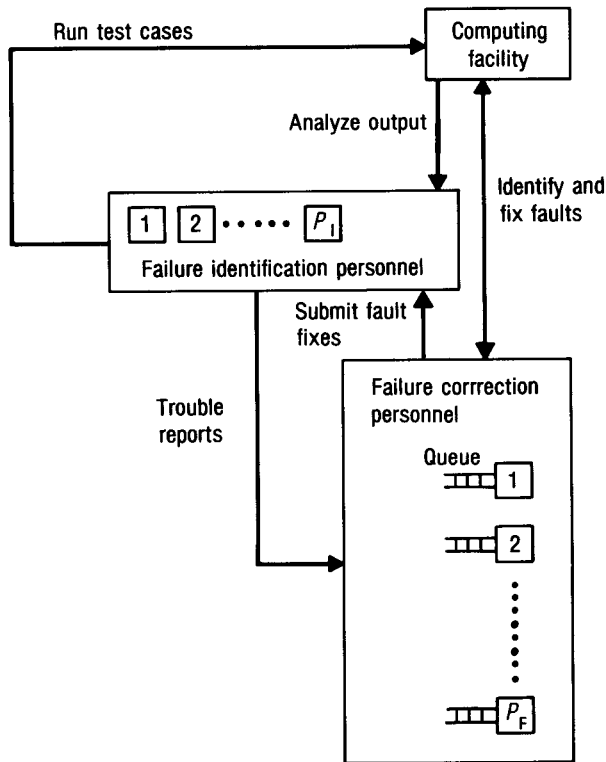


FIG. 8. Failure queuing diagram for the system test phase.

The failure-identification personnel are the members of the test team. They run test cases and compare the results against program requirements to establish what failures have occurred. They also generate a trouble or failure report for each failure identified, giving all the pertinent facts. They do not search for the fault causing a failure. The work of test planning and test case development is usually completed before testing, thus not ordinarily burdening the failure-identification personnel when that resource governs the pace of testing. We will denote the number of *available* failure-identification personnel by P_I . The failure-identification personnel do not necessarily devote work full time to testing. "Available" only means that they be able to do so given sufficient notice.

The failure-correction personnel are the debuggers or developers available to remove faults from the software. They determine the fault that caused a failure, remove it, and prove that the failure no longer occurs. Correction includes preparing change documentation during (not after) the test period. We will denote the number of *available* failure-correction personnel by P_F . "Available" is used with the same meaning as for the failure-identification personnel.

Computer time represents the computer facilities necessary for the failure-identification personnel and failure-correction personnel to do their jobs. We define computer time as the measure that is used for allocating computer resources. We will denote the available computer time by P_C . This quantity is measured in terms of prescribed work periods to put it on the same basis as personnel. The prescribed work period is the standard work week for which personnel are normally available. It can be greater than 40 hours if overtime is standard.

At any time the amount of work being performed by one of the resources uses that resource to the fullest. That resource is called the *limiting resource*. It determines how much calendar time is being used with respect to execution time. The limiting resource concept is the basis for calendar-time component modeling.

The following is a common scenario. At the start of testing the test team identifies a large number of failures separated by short time intervals. Testing must be stopped from time to time to let the debuggers keep up with the load. As testing progresses, the intervals between failures become longer and longer. The time of the debuggers is no longer completely filled with failure correction work, but now the test team becomes the bottleneck. The effort required to run tests and analyze the results is occupying all their time. That paces the amount of testing done each day. Finally, at even longer intervals, the capacity of the computing facilities becomes limiting. This resource then determines how much testing is accomplished.

The calendar-time component is based on a "debugging process" model.

This model takes into account

1. the resources *used* in running the program for a given execution time and identifying and processing an associated quantity of failures,
2. the resource quantities *available*, and
3. the degree that a resource can be *utilized* (due to bottlenecks) during the period that it is limiting.

To simplify the modeling process, the quantities of the available resources are assumed constant from the point of use of the model and through the remainder of the test period. It is possible to handle more complex patterns of resource availability. However, it does not seem to be worth the effort, since

1. Constant or nearly constant resources during test are a common condition.
2. The constancy assumption must hold only for the period of prediction. Resource values can be changed for the next prediction.
3. If resources are not constant, average values give results that are a good first approximation to reality.

2.4.2 Model of Resource Usage

To relate calendar time to execution time one needs to know how resource requirements are related to execution time. Resource requirements can be approximated by a model of the form

$$\chi_r = \theta_r \tau + \mu_r \mu(\tau), \quad (2.12)$$

where r denotes a resource ($r = C, F,$ or I for computer time, failure-correction personnel, or failure-identification personnel, respectively), χ_r is the resource requirement, θ_r is the average resource usage rate with execution time, μ_r is the average resource usage per failure, τ is the execution time, and $\mu(\tau)$ is the mean-value function or expected number of failures by time τ . Resource usage represents a resource applied for a time period, for example, one person hour. Since the mean-value function is a function of execution time the resource usage given by Equation (2.12) is actually a function of execution time only. It is also dependent on the execution-time component of the software reliability model being used. Thinking in terms of expected number of failures *and* execution time is useful in gaining physical insight into what is happening and allows model-independent resource usage parameters to be determined.

Consider a test team that is working with a particular piece of software. If they conduct two hours of testing rather than one, they will have twice the amount of work to do. There will be about twice the amount of output to

examine, and they will spend twice as much time conducting the tests. The work effort related to failures experienced will be dependent on the number. A failure report must be written whenever a failure occurs. Some time is needed to check the requirements to determine whether each suspected failure really is one. The test team may have to record some information, and perhaps make a couple of runs to get all the pertinent facts about the particular failure. Note that mean failures is used to estimate effort, so that mean resource requirements are obtained.

For failure correction, required resources depend only on the mean failures experienced. However, computer time is expended for both identification and correction of failures. Thus, computer time used depends on the amount of execution time and the number of failures.

Frequently it is necessary to work with the incremental or additional resource requirement $\Delta\chi_r$. This is given by

$$\Delta\chi_r = \theta_r \Delta\tau + \mu_r \Delta m(\tau), \quad (2.13)$$

where $\Delta\tau$ is the incremental testing time and $\Delta m(\tau)$ is the incremental number of expected failures.

While the preceding arguments make Equation (2.12) or Equation (2.13) plausible, the true test is how well the model agrees with actual data. For this purpose, data on failure-identification work, failure-correction work, and computer time have been collected on four projects. One set of such data is shown in Table III. The data comes from a real-time command and control system developed by nine programmers and consisting of 21,700 delivered object instructions. A total of 136 failures were experienced in 24.6 CPU hours or 92 calendar days. The data was taken from Musa (1979a). The data was collected weekly from each project member. Each person recorded his or her resource usage for the previous week in the categories shown in Table III. Data was also recorded from project logs on execution time (in hours) for each week.

Estimates of the parameters θ_r and μ_r for each resource for each project are obtained using linear least squares fits. For the following results to be model independent it is necessary to approximate the value of the mean-value function by the observed number of failures $m(\tau)$. Thus each incremental resource requirement is given approximately by

$$\Delta\chi_r = \theta_r \Delta\tau + \mu_r \Delta m(\tau). \quad (2.14)$$

This can be converted into a simpler form by dividing both sides by $\Delta\tau$. Thus,

$$\frac{\Delta\chi_r}{\Delta\tau} = \theta_r + \mu_r \frac{\Delta m(\tau)}{\Delta\tau}. \quad (2.15)$$

As a result of this transformation there is only one independent variable, $\Delta m(\tau)/\Delta\tau$, and a new dependent variable, $\Delta\chi_r/\Delta\tau$. The number of failures

TABLE III
INCREMENTAL RESOURCE USAGE DATA—SYSTEM T1

Week	Execution time, $\Delta\tau$ (CPU hr)	Failure identification work, $\Delta\chi_i$ (person hr)	Failure correction work, $\Delta\chi_F$ (person hr)	Computer time, $\Delta\chi_C$ (CPU hr)	Failures identified	Failures corrected
1	0.00917	2.0	5.0	4.0	2	1
2	0.010	1.5	6.0	4.3	0	1
3	0.003	1.0	3.5	2.0	0	0
4	0.023	0.5	2.3	0.6	1	1
5	0.041	1.0	6.5	2.3	1	1
6	0.004	0.5	2.7	1.6	2	0
7	0.025	0.5	3.1	1.8	1	1
8	0.302	4.0	21.5	14.7	9	2
9	0.973	15.0	87.5	25.1	13	6
10	0.020	2.0	16.0	4.5	2	4
11	0.450	8.5	43.0	9.5	11	1
12	0.250	3.5	67.0	8.5	2	14
13	0.94	16.0	103.0	29.5	11	5
14	1.34	32.5	103.1	22.0	14	19
15	3.32	42.0	66.0	39.5	18	19
16	3.56	22.0	92.7	26.0	12	10
17	2.66	17.0	105.5	25.5	12	12
18	3.77	18.5	75.1	31.4	15	20
19	3.40	17.0	67.5	30.0	6	12
20	2.40	13.0	33.0	12.8	3	2
21	1.80	9.0	4.6	5.0	1	5
Total	25.3	227	914.6	300.6	136	136

identified is used for $m(\tau)$ in fitting the failure-identification work, while the number corrected is used in fitting the failure-correction work. The average of both these numbers is used for the computer work since the computer work involves both failure identification and correction.

A statistical analysis demonstrates that the θ_F term for failure-correction work is 0. The invariance of failure-correction work with execution time is also supported by data taken by Shooman and Bolsky (1975). One might believe that failures occurring late in a test phase are more complex and hence require more effort to correct. Apparently this hypothesis is false or there is a countervailing factor. One such factor might be increasing programmer experience with the system.

Resource parameter values resulting from the least squares fit using Equation (2.15) for the four systems are given in Table IV. The θ_F term was set

TABLE IV
RESOURCE PARAMETER VALUES

Resource usage coefficients	System			
	T1	T2	T3	T4
μ_C (CPU hr/failure)	1.56	1.73	0.38	0.63
μ_F (person hr/failure)	4.42	6.88	0.58	0.68
μ_I (person hr/failure)	0.23	3.89	0.53	0.24
θ_C	4.80	1.73	1.06	2.13
θ_I (person hr/CPU hr)	9.92	3.24	4.77	5.70

equal to zero in fitting the equation for failure-correction work due to the foregoing results. Both terms were used for the other two resources. It is believed that the differences in parameter values are mainly the result of the different skill levels of the teams involved (the largest team had nine people).

The resource usage model, when used with the estimated parameters, yields good agreement with the data. For example, Figs. 9–11 illustrate the fit between the model of resource usage and the data from Table III.

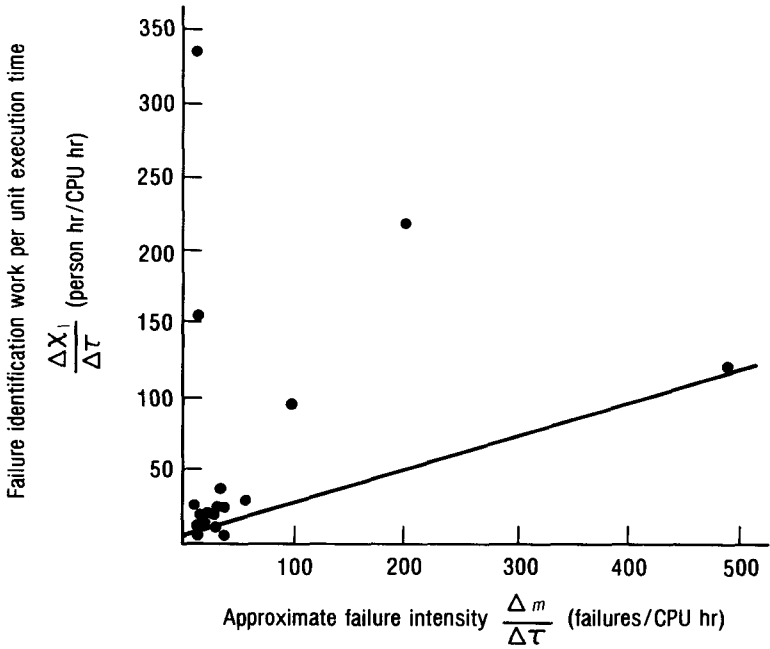


FIG. 9. Failure-identification work for system T1-model versus actual.

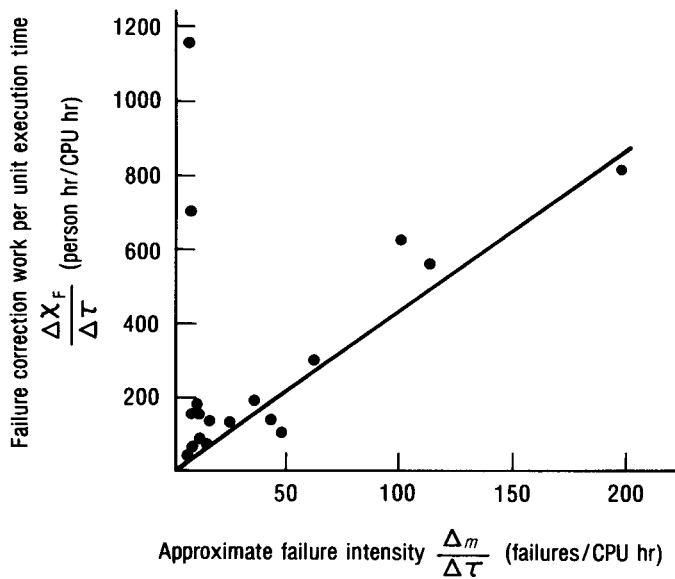


FIG. 10. Failure-correction work for system T1-model versus actual.

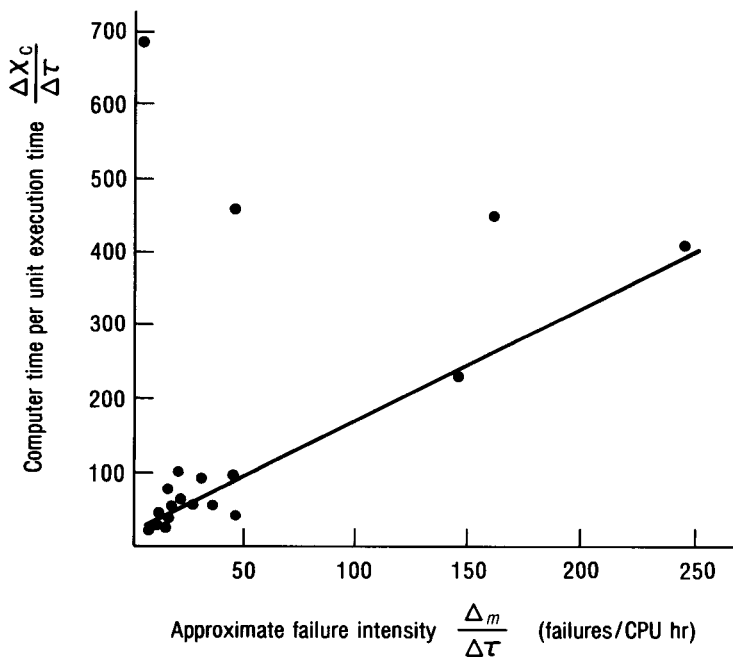


FIG. 11. Computer time for system T1-model versus actual.

2.4.3 Calendar-Time Relationship to Execution Time

The rate of passage of calendar time with respect to execution time is related to the rate of resource usage with respect to execution time for a given limiting resource through the relation

$$\frac{dt_r}{d\tau} = \frac{1}{P_r \rho_r} \frac{\partial \chi_r}{\partial \tau}, \quad (2.16)$$

where $dt_r/d\tau$ represents the instantaneous ratio of calendar time to execution time, and ρ_r is the utilization of the r th resource. Note that $P_r \rho_r$ represents the *effective* amount of resource r that is available. A discussion of resource utilization is given in Section 2.4.4. At any point in execution time one resource is used at its limit. This resource is the one that gives the largest value for the above ratio. Thus the passage of calendar time is determined by the equation

$$\frac{dt}{d\tau} = \max_r \left(\frac{dt_r}{d\tau} \right). \quad (2.17)$$

Substituting Equations (2.12) and (2.16) into Equation (2.17) yields

$$\frac{dt}{d\tau} = \max_r \left\{ \frac{1}{P_r \rho_r} [\theta_r + \mu_r \lambda(\tau)] \right\}. \quad (2.18)$$

Figure 12 illustrates a typical form of the calendar-time component. The instantaneous ratio of calendar time to execution time versus execution time for each resource is plotted. The ratio always has an initial value of $(\mu_r \lambda_0 + \theta_r)/P_r \rho_r$ and always approaches the asymptote given by $\theta_r/P_r \rho_r$. The thick dark curve represents the maximum value of the ratio. It is the one that governs the passage of calendar time.

The point, in terms of failure intensity, where a potential transition occurs from one limiting resource r to another s , may be determined by equating $dt_r/d\tau$ and $dt_s/d\tau$. The potential transition points are given by

$$\lambda_{rs} = \frac{P_s \rho_s \theta_r - P_r \rho_r \theta_s}{P_r \rho_r \mu_s - P_s \rho_s \mu_r}, \quad r \neq s. \quad (2.19)$$

Once again refer to Fig. 12. The potential transition points given above were converted to execution times, that is, the potential transition between resource r and s occurs at time τ_{rs} (which corresponds to a failure intensity of λ_{rs} —note that λ_{rs} is independent of the specific execution-time model used while τ_{rs} is not). Limiting resources are readily identified as the failure-correction personnel for $0 \leq \tau \leq \tau_{FI}$, the failure-identification personnel for $\tau_{FI} \leq \tau \leq \tau_{IC}$, and computer time for $\tau \geq \tau_{IC}$. The term “potential” transition is used because an actual transition does not occur, unless one of the resources of Equation (2.19) is limiting. In the figure the potential transition at τ_{FC} does not occur

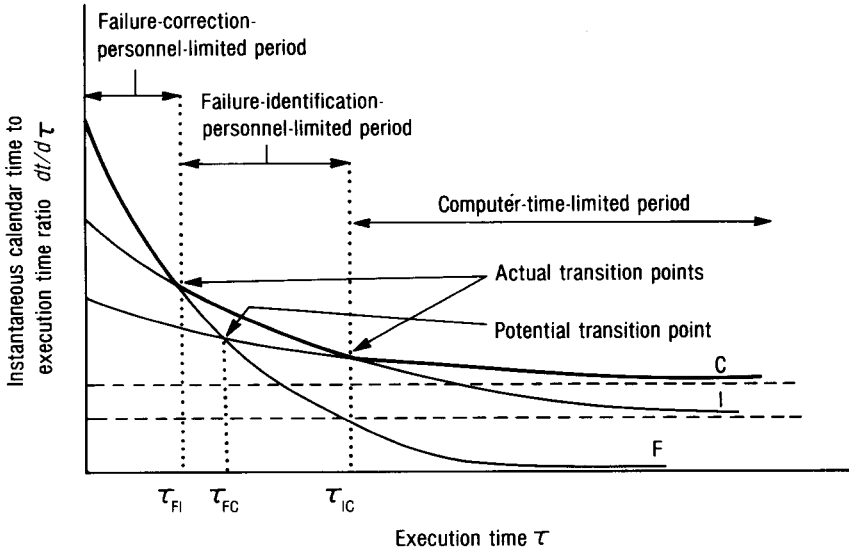


FIG. 12. Typical execution-time relationship among resource usages.

because neither resource F nor C is limiting at that point. This case is depicted in terms of failure intensities (rather than execution time) in Fig. 13. This represents the most typical case encountered in practice.

It can be readily shown that the increment of calendar time over a segment made up of just one limiting resource is given by

$$\Delta t_r = \int_{\tau_1}^{\tau_2} \frac{\theta_r + \mu_r \lambda(\tau)}{P_r \rho_r} d\tau, \quad (2.20)$$

where τ_1 and τ_2 define the boundaries of the region for which the resource is limiting.

For the case shown in Fig. 13 the total calendar time required to achieve a failure intensity of λ_F is

$$\Delta t = \Delta t_F + \Delta t_I + \Delta t_C = \sum_r \Delta t_r, \quad (2.21)$$

where Δt_F , Δt_I , and Δt_C are the increments of t corresponding to the change in failure intensity from λ_0 to λ_{FI} , λ_{FI} to λ_{IC} , and λ_{IC} to λ_F , respectively. A similar expression holds if system test had already started and a failure intensity of λ_p was reached. In the case shown Δt_F would be the increment of t corresponding to the change in λ from λ_p to λ_{FI} , the other terms of Equation (2.21) remaining

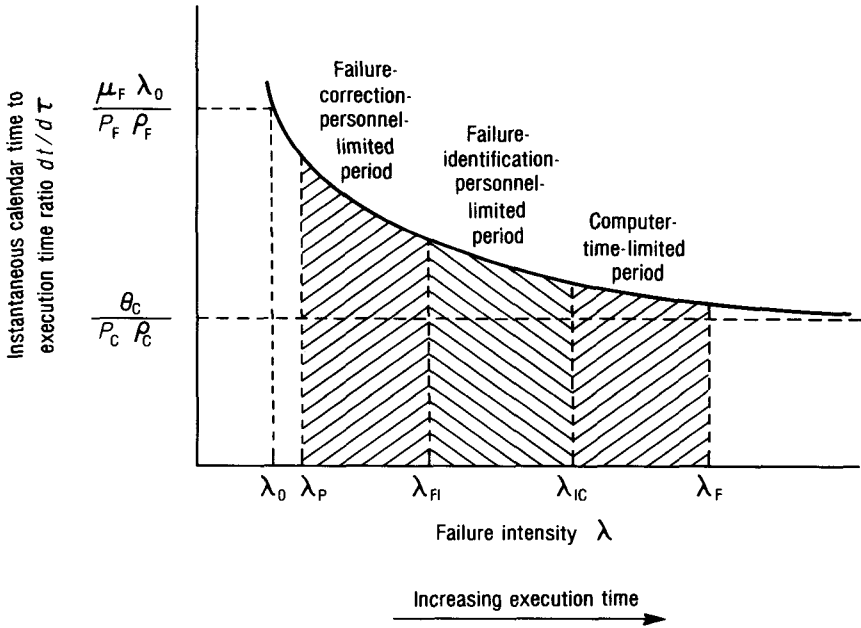


FIG. 13. Typical failure intensity relationship among limiting resources.

unaltered. In practice, many resource transitions are possible, and it would be tedious to list them all. However, they are all treated like the preceding case.

In computing calendar time, note that scheduled idle periods during which the limiting resource is not available must be accounted for. This is handled by specifying lengths of workday (hours/per day) and workweek (days/per week).

2.4.4 Resource Utilization

To fully specify the calendar-time component, the utilization of each resource during the period that it is limiting needs to be determined. These utilizations are *maximum* figures that apply during the period that the resource is limiting. At other times the resource utilization is lower. The easiest of these utilizations to establish is that for the failure-identification personnel. Here ρ_1 is set equal to 1 since utilization of this resource is not restricted by any queueing constraints. There is ordinarily no reason for available failure-identification personnel not being fully used.

Determination of failure-correction personnel and computer time utilizations is somewhat more complicated. Each is discussed in turn in the next two sections.

Failure-Correction Personnel. Available failure-correction personnel cannot be fully used during the failure-correction-personnel-limited period because of the unpredictable identification times of failures and inequality in load among the debuggers. Projects appear to naturally prevent an excessive backlog of failures awaiting correction from building up in any area of code by limiting utilization of failure-correction personnel. Such a backlog could hamper further testing in that area. In measuring backlog, failures whose correction is deferred for reasons other than availability of personnel to fix them are not counted. Note that the area of code is generally associated with one responsible person.

The queueing constraints on each debugger are modeled as a classical queueing system with random arrivals and random service. Both identification and correction of failures are considered to be time-invariant Poisson processes in calendar time. Each process (identification and correction) will be considered in detail to demonstrate that this is a satisfactory representation of reality.

Use of the Poisson process to represent the correction of failures has two implications. First, the correction of each failure is independent of the correction of other failures. Second, the probability that a fault is removed during any time period of debugging effort is equal to the probability that it is removed in any other such time period. The latter is a good assumption, although it may overestimate the proportion of failures with short fix times.

The Poisson failure-correction process also implies that the failure-correction effort (in person hours) is exponentially distributed. To determine whether the exponential distribution is likely to be an acceptable model for the distribution of failure-correction times, consider the data shown in Fig. 14. The data (representing 178 failure corrections) were taken from the unit debugging phase of a project and involved eight programmers. All times are "direct effort" times, that is, no overhead factor has been applied to account for absence, training, meetings, coffee breaks, etc. The failure-correction time distribution is compared with an exponential distribution with the same mean (1.185). Visual inspection indicates good agreement. A chi-square goodness-of-fit test shows that the data are consistent with the assumption that they come from an exponential distribution.

In system testing, a constant component will probably be added to the distribution of time given for unit testing. This component, which represents administrative overhead and retest requirements, is usually smaller than the mean time for the variable component (Herndon and Keenan, 1978). It is not likely to have any appreciable effect on the relationship between utilization and queue length.

For failure identification, the Poisson process implies a constant arrival rate in calendar time for failures needing to be fixed. It also requires a random assignment of failures among the failure-correction personnel.

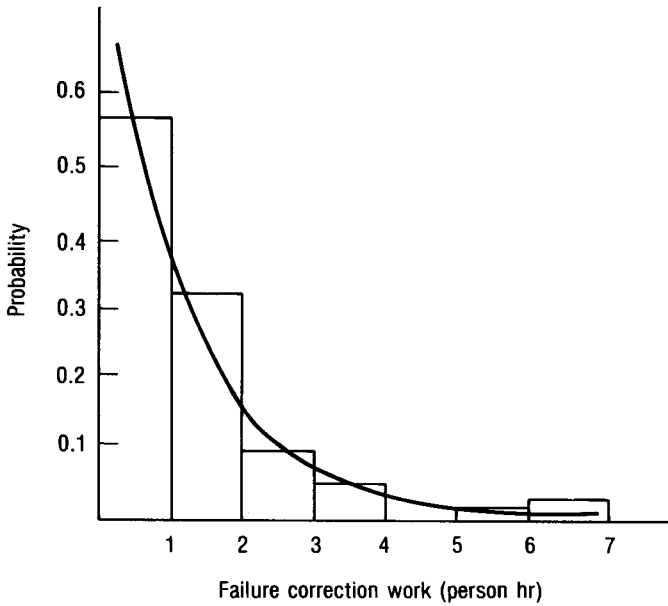


FIG. 14. Exponential distribution of failure-correction times.

The random-assignment assumption is reasonable since usually no section of code (and the programmer responsible for it) is favored during test. If an evolving program is under test, random assignment can still occur as long as each debugger has some code in the part of the system being tested. If not, this case can still be handled by applying an appropriate correction based on the debuggers who are involved. For example, the number of debuggers can be equated to the average number to whom failures can be assigned.

A constant arrival rate of failures with respect to calendar time during the failure-correction-personnel-limited period can be shown as follows. Let $\lambda(t)$ denote the failure intensity with respect to calendar time. Now $\lambda(t)$ can be obtained from the derivative of the mean-value function (expressed in calendar time) with respect to calendar time, that is,

$$\lambda(t) = \frac{d\mu(t)}{dt}. \quad (2.22)$$

During the failure-correction-personnel-limited period t and τ are related by Equation (2.18) with $\theta_F = 0$, that is,

$$\frac{d\tau}{dt} = \frac{P_F \rho_F}{\mu_F} \frac{1}{\lambda(\tau)}, \quad (2.23)$$

where the differential is inverted. Using the chain rule yields

$$\lambda(t) = \frac{d\mu(\tau)}{d\tau} \frac{d\tau}{dt}. \quad (2.24)$$

The first differential is the failure intensity with respect to execution time $\lambda(\tau)$. Substituting Equation (2.23) into Equation (2.24) results in

$$\lambda(t) = \lambda = \frac{P_F \rho_F}{\mu_F}. \quad (2.25)$$

This is constant if ρ_F is (possibly maintained) constant. This finding has been independently verified by Ehrlich and Ejzak (1988).

Debugger utilization can now be determined. The arrival rate of failures needing correction to each debugger is λ/P_F . The correction service rate of each debugger is $1/\mu_F$. The utilization factor is defined (in queueing theory) as the ratio of arrival rate to service rate, or

$$\rho_F = \frac{\lambda \mu_F}{P_F}. \quad (2.26)$$

The probability that a debugger has a queue of m_Q or more failures awaiting correction or being corrected is $\rho_F^{m_Q}$ in the steady state (see $M/M/1$ queueing system in Kleinrock (1975)). The probability P_{m_Q} that no debugger has m_Q or more failures queued is

$$P_{m_Q} = (1 - \rho_F^{m_Q})^{P_F}. \quad (2.27)$$

Hence, no debugger has m_Q or more failures to work on at any given time with probability P_{m_Q} if ρ_F is limited to

$$\rho_F = (1 - P_{m_Q}^{1/P_F})^{1/m_Q}. \quad (2.28)$$

If ρ_F is controlled at the preceding value, then effective manpower, under the constraint of preventing excessive backlogs, is maximized.

Using the steady-state value of probability for queue length results in slightly overestimating the time expended, as queues are shorter during the buildup transient. However, the identification and correction of failures were also assumed to be parallel processes. Since correction always follows identification, there are intervals at the start and the end of the test phase during which only one process is occurring. This assumption results in a small underestimate of calendar time expended. The two preceding factors can be assumed to approximately cancel.

Figure 15 gives curves of utilization versus failure-correction personnel for various values of queue length. The curves are constructed so that 90% of the

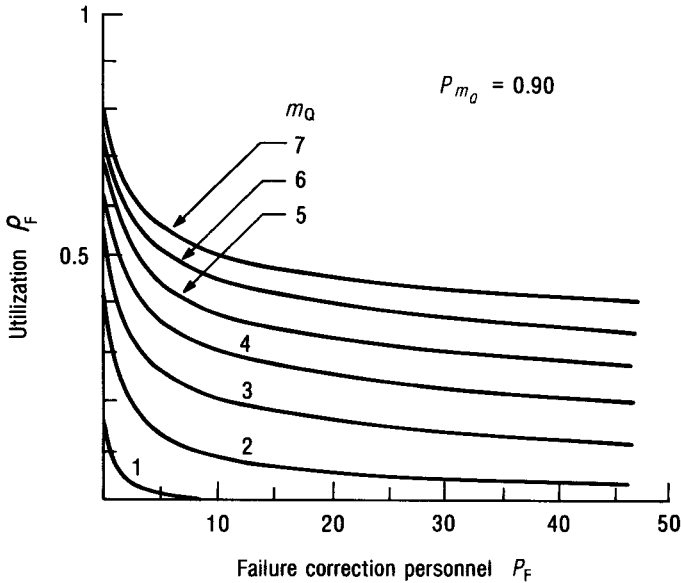


FIG. 15. Failure-correction personnel utilization.

queues that occur are less than the specified value. The topic of what queue length is most commonly encountered is a matter for further research; the best current estimate is 5.

Computer Time. The method for determining the computer utilization factor ρ_C depends on whether turnaround time or response time can be controlled or not. In the case of a small project using a general-purpose computation facility, it usually cannot. Here ρ_C should be set to its measured value. If wait time is much greater than usual and it cannot be profitably used in other activities, then μ_F should be increased by the ratio of correction work time plus excess wait time to correction work time. In the case of a larger project that uses a major portion of the computer's load, ρ_C may be controllable. If so, then its value should be set to obtain "good" turnaround or response. Response is probably most important, since almost all failure-identification and correction activity is interactive. Frequently utilization is controlled by providing sufficient hardware to maintain it at an economic optimum. The sum of computing costs and wait time (for response) costs of software engineers is minimized. When utilization is high, response is poor and salary costs (due to waiting) are high, although direct computing costs are low. When utilization is low, response is excellent and salary costs are low but,

direct computing costs are high. Minimum overall cost occurs at some intermediate value of utilization.

2.5 Selected Model

The model selected for presentation in the remainder of this article is the basic execution-time model of Musa (1975). It has an execution-time component and a calendar-time component. Each component will be described along with the determination of its parameters.

There are six primary reasons for selecting the basic execution-time model.

1. It generally predicts satisfactorily.
2. It is simple and easy to understand.
3. It is the model that is most thoroughly developed and has been most widely applied to actual projects.
4. Its parameters have a clear physical interpretation that can be related to information existing before the program has been executed. The information includes characteristics of the software such as size and characteristics of the development environment. This property is important for system engineering studies.
5. It can handle evolving systems (systems that change in size) by adjusting the failure times to estimates of what they would have been if all the code had been present (Musa *et al.*, 1987).
6. It readily yields useful quantities, such as the date for attaining a failure intensity objective.

In evaluating the basic execution-time model, Dale (1982) states, "This is arguably the most practical model of all. It is a statistical model, and has been developed to the extent that it takes into account various personnel and resource constraints." Ramamoorthy and Bastani (1982) note that the Musa model "is better developed and is the first model to insist on execution time."

Managers in projects that have applied the model have noted benefits in addition to having quantitative measures. They observed that the model provides an intuitively appealing, well-structured conceptual framework that gives a new perspective on the testing process. Developers and their customers were stimulated to consider carefully what behaviors should be considered failures. Also, they gave more careful thought to what the reliability objective should be.

Before leaving the topic of model selection, let us note why we did not choose a number of other models. First, some of the other models do not use the execution-time component calendar-time component approach that has proved so fruitful. Either reliability modeling is performed in calendar time,

resulting in severe disadvantages, or the time domain is not specified. If an unspecified model is interpreted in terms of execution time, there may not be a way of relating execution time to calendar time, although one might possibly generalize the approach of Section 2.4. The calendar-time capability is very important for engineers and managers. Most models do not provide many of the useful quantities that the basic execution-time model does, such as the calendar time required to reach a failure intensity objective. Concepts from several of the models (Goel and Okumoto, 1979b; Jelinski and Moranda, 1972; Shooman, 1972) have been incorporated in the basic execution-time model. Thus, when applying the basic model, one can enjoy most of the advantages of the preceding models and those of the basic model itself. Some models, such as the Littlewood and Verrall model (Littlewood and Verrall, 1973), are very complex. They are difficult to understand and visualize physically. Also, they are relatively difficult and costly to implement on a computer for practical and economical use. Finally, none of the other models have been applied as extensively as the basic execution-time model, with the result that they do not have the extensive practical knowledge that has been gained from it.

Experience indicates that the basic model can be applied to any type or size of software project, with the following exception. Very small projects (less than about 5,000 lines of code) may not experience sufficient failures to permit accurate statistical estimation of execution-time component parameters and hence the various derived quantities. This does not mean that nothing can be done. Some kind of estimate can usually be made, such as a result of the form "present failure intensity is at least X failures per CPU hour."

Each of the two components of the basic model is considered here. The component itself is described along with the determination of its parameters.

2.5.1 Execution-Time Component

Although not originally described as such, the basic execution-time model is best characterized as a finite-failures Poisson-type model. Goel and Okumoto (1979b) described this kind of model in terms of a nonhomogeneous Poisson process.

The basic model assumes that the time to failure of an individual fault has an exponential distribution, that is, $f_a(\tau) = \phi \exp(-\phi\tau)$. The per-fault hazard rate is constant at ϕ , that is, $z_a(\tau) = \phi$. Musa (1975) described ϕ , using additional elements, as

$$\phi = fK, \quad (2.29)$$

where f is the linear execution frequency of the program or the ratio of average instruction rate to program size in object instructions, and K is the

fault exposure ratio. This ratio accounts for the effects of program dynamic structure and data dependency on the hazard rate.

For a given fault reduction factor B , the expected number of failures experienced by time τ is

$$\mu(\tau) = v_0[1 - \exp(-\phi B\tau)]. \quad (2.30)$$

This relationship is illustrated in Fig. 16. The expected number of failures is finite as time approaches infinity. This is generally true for the basic model during testing, although it is ordinarily infinite during the operational phase.

The failure intensity is found by differentiating Equation (2.30) to yield

$$\lambda(\tau) = v_0\phi B \exp(-\phi B\tau). \quad (2.31)$$

This is an exponentially decaying function of τ with the initial failure intensity $\lambda_0 = v_0\phi B$. This relationship is illustrated in Fig. 17.

The failure intensity λ for the basic model as a function of failures experienced is

$$\lambda(\mu) = \phi B(v_0 - \mu). \quad (2.32)$$

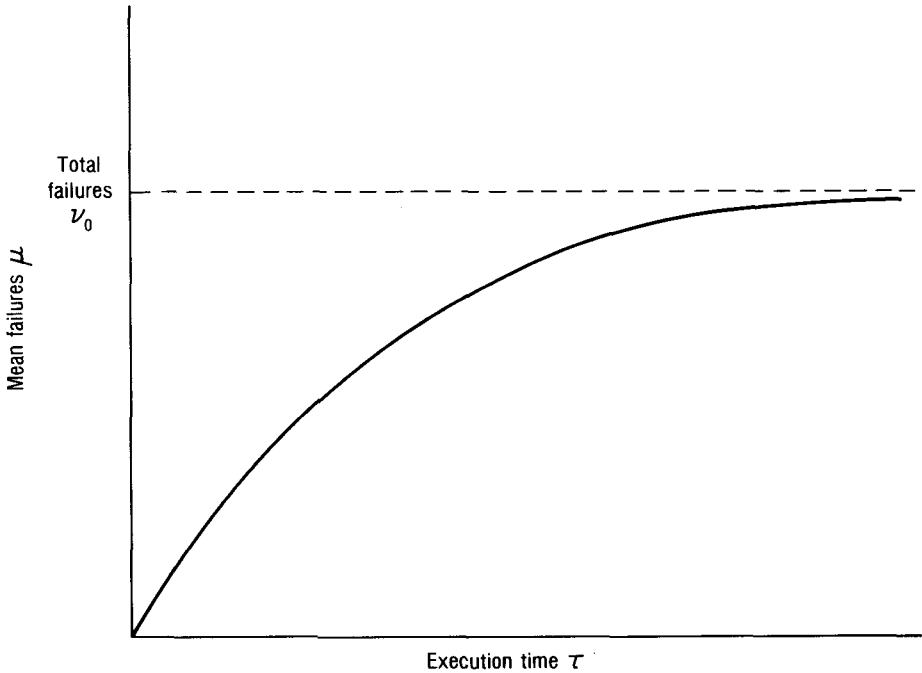


FIG. 16. Expected failures experienced for basic execution-time model.

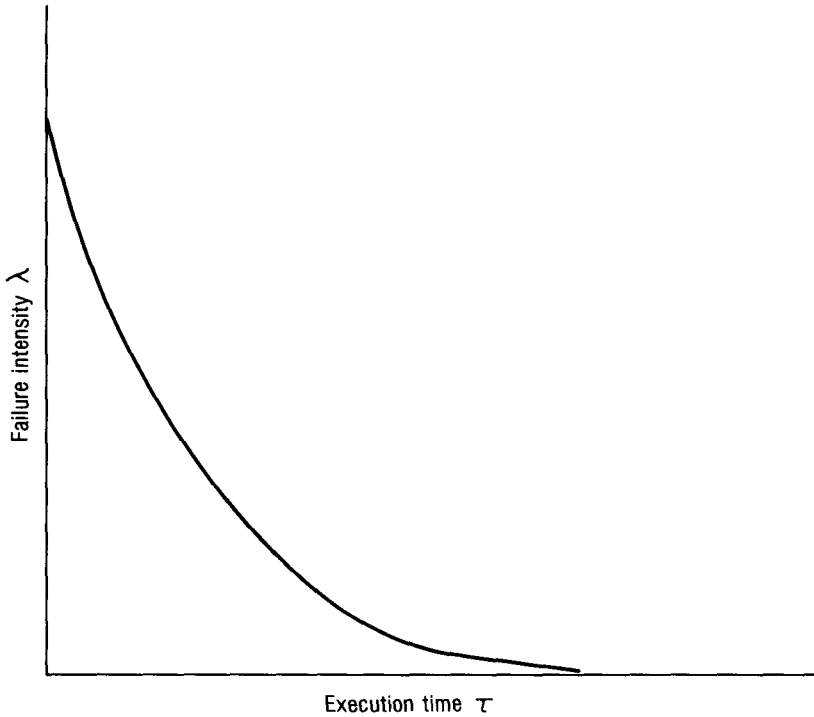


FIG. 17. Failure intensity for basic execution-time model.

The slope of failure intensity, $d\lambda/d\mu$, is given by

$$\frac{d\lambda}{d\mu} = -\phi B = -\frac{\lambda_0}{v_0}. \quad (2.33)$$

If a failure intensity objective λ_F has been set for the program, then the additional number of failures that must be experienced to meet this objective can be derived from Equation (2.32) as

$$\Delta\mu = \frac{v_0}{\lambda_0} (\lambda - \lambda_F). \quad (2.34)$$

Similarly, using Equation (2.31), the required additional execution time is

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln \left(\frac{\lambda}{\lambda_F} \right). \quad (2.35)$$

2.5.2 Calendar-Time Component

Equations (2.34) and (2.35) are of interest in themselves. The additional expected number of failures required to reach the failure intensity objective gives some idea of the failure-correction workload. The additional execution time indicates the remaining amount of testing required. However, even more significantly, they are both used in making estimates of the additional calendar time required to reach the failure intensity objective.

Specification of the calendar-time component is complete once Equation (2.20) is evaluated. Carrying out this evaluation yields

$$\begin{aligned}\Delta t_r &= \frac{1}{P_r \rho_r} \{ \theta_r (\tau_2 - \tau_1) + \mu_r [\mu(\tau_2) - \mu(\tau_1)] \} \\ &= \frac{1}{P_r \rho_r} (\theta_r \Delta \tau + \mu_r \Delta \mu).\end{aligned}\tag{2.36}$$

3. Application of Software Reliability

We will consider four groups of applications (not necessarily an exhaustive set), each of which will be treated in more detail:

1. system engineering,
2. project management during development and particularly testing,
3. operational-phase software management, and
4. evaluation of software engineering technologies.

Table V summarizes some of the applications and evaluates their potential value and current quality of results. Before dealing with the applications, however, we will examine three types of calculations common to many of them to gain some insight. These are computation of

1. duration of system testing,
2. system test resource requirements and cost, and
3. operational cost of failures.

When we compute costs, we consider only the reliability-dependent components, not total costs. Other costs can be calculated and added if total costs are needed. Many applications, however, are concerned principally with determining the reliability that minimizes cost, so that only the reliability-dependent component is needed. "Reliability dependent" refers to the differences in reliability obtained by varying the amount of testing. Of course,

TABLE V
SUMMARY AND EVALUATION OF APPLICATIONS

Development phase	Applications	Potential value of application	Current quality of results
System engineering	Product feasibility studies	Very high	Fair –
	Tradeoff studies	High +	Fair +
	Cost studies	High	Fair
	Scheduling studies	High +	Fair
	Reliability allocation	High –	Fair +
	Reliability specification	High –	Fair
Test	Status tracking	High	Good +
	Project completion estimation	High	Good –
	Redoing system engineering studies	High	Good
	Investigating management alternatives	High	Good
Operation	Reliability demonstration	High –	Excellent
	Change control	High	Excellent
	Preventive maintenance control	High –	Excellent
All	Evaluating software engineering technology	High –	Excellent

reliability is affected by the support environment and methodologies used on a project, but these factors are not easily varied while the project is under way.

The time required, resources needed, and costs associated with system testing are principally dependent on the initial reliability at the start of system testing and the reliability goal to be attained. Although program size influences the two foregoing quantities, it is definitely not the sole factor. Hence estimating “person months per K lines of code” is not generally appropriate, and a reliability-related approach must be taken. The procedure for calculating system test duration and system cost is, for the most part, independent of the particular execution-time component used. For illustration, the basic execution-time model is used.

3.1 General Computations

3.1.1 Duration of System Testing

In order to compute system test duration (excluding the test planning effort), we must first determine the values of the execution-time component parameters.

The calendar-time interval t consists of the sum of one to three periods. In each period, a different resource is *limiting*. This means that the resource produces the maximum ratio of calendar time to execution time for that period. The duration of each period is computed separately, based on its limiting resource. Then the durations are added, yielding

$$t = \sum_r \frac{\Delta\chi_{rr}}{P_r \rho_r}. \quad (3.1)$$

The quantity $\Delta\chi_{rr}$ is the requirement for resource r during the period that it is limiting. For example, it might be 100 person days. It is generally not the same as the total resource requirement to meet the failure intensity objective $\Delta\chi_r$. Note that P_r is the quantity of resource r that is *available* (for example, five persons). The quantity ρ_r is the utilization factor for resource r . The resource utilization factor reflects the possibility that all of an available resource cannot be usefully employed.

Each resource requirement during its limiting *period* is given by

$$\Delta\chi_{rr} = \theta_r \Delta\tau_r + \mu_r \Delta\mu_r. \quad (3.2)$$

Note that θ_r is the resource expenditure per unit execution time. The quantity μ_r is the resource expenditure per failure. The symbol $\Delta\tau_r$ represents the execution time interval for the *period*. The quantity $\Delta\mu_r$ is the number of failures experienced in the *period*.

To obtain the expected number of failures experienced in the period for the basic model,

$$\Delta\mu_r = \frac{v_0}{\lambda_0} (\lambda_{r_1} - \lambda_{r_2}) \quad (3.3)$$

is used (compare with Equation (2.34)). The quantities λ_{r_1} and λ_{r_2} are the failure intensities at the boundaries of the limiting resource period.

The execution time interval is determined from (compare with Equation (2.35))

$$\Delta\tau_r = \frac{v_0}{\lambda_0} \ln \left(\frac{\lambda_{r_1}}{\lambda_{r_2}} \right). \quad (3.4)$$

The boundaries of the different resource-limited periods λ_{r_1} and λ_{r_2} are the present and objective failure intensities and the transition points λ_{rs} (derived in Section 2) that lie within the range. The potential transition points are obtained from Equation (2.19) where the subscripts (r, s) have the values (F, C), (F, I), and (I, C). One must determine which resource-limited periods actually occur from an examination of the boundaries and a determination of which ratio of calendar-time to execution-time is maximum at any point in each

period:

$$\left. \frac{dt}{d\tau} \right]_{\max} = \max \left(\frac{\theta_r + \mu_r \lambda}{P_r \rho_r} \right). \quad (3.5)$$

The quantity λ is any failure intensity in the range $[\lambda_{r1}, \lambda_{r2}]$.

3.1.2 System Test Resource Requirements and Cost

To compute resource requirements and cost, we start with the determination of the values of the execution-time component parameters. We then determine the expected number of failures

$$\mu = v_0 \left(1 - \frac{\lambda_F}{\lambda_0} \right) \quad (3.6)$$

that must be experienced and the associated execution time,

$$\tau = \frac{v_0}{\lambda_0} \ln \left(\frac{\lambda_0}{\lambda_F} \right), \quad (3.7)$$

to decrease the failure intensity from λ_0 to the failure intensity objective λ_F (see Equations (2.34) and (2.35)). Normalized values of these quantities have been plotted in Figs. 18 and 19 as a function of the failure intensity ratio λ_0/λ_F . In Fig. 18, we plot the fraction μ/v_0 of total failures that must be experienced to reach the failure intensity objective. As the failure intensity ratio approaches infinity, the fraction of total failures that must be experienced approaches 1. In Fig. 19, a normalized execution time increment $B\phi\tau$ is plotted, where B is the fault reduction factor and ϕ is the per-fault hazard rate. As the failure intensity ratio approaches infinity, the normalized (and hence actual) execution time increment approaches infinity. Each of the three *total* resource expenditures χ_r is given by

$$\chi_r = \theta_r \tau + \mu_r \mu. \quad (3.8)$$

As the failure intensity ratio approaches infinity, the failure-identification and computer resource expenditures approach infinity logarithmically. The general shape of the curves is illustrated in Fig. 20. The failure-correction resource expenditure approaches the product of total failures and failure correction effort per failure. The curve's general shape is the same as that shown in Fig. 18. To obtain overall cost, we multiply the χ_r by cost rates and total the results. Cost approaches infinity as the failure intensity ratio approaches infinity. The general shape of the cost curve is shown in Fig. 20. In the case of large failure intensity ratios, the cost curve approaches a logarithmic shape. Suppose you decrease the failure intensity objective or increase the failure intensity ratio by a factor of two. The cost of reaching the

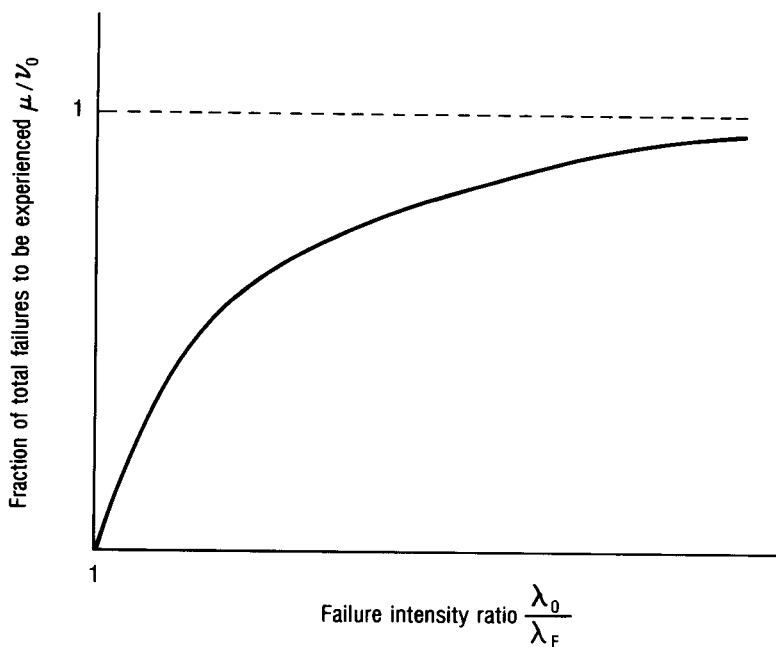


FIG. 18. Variation of fraction of total failures to be experienced with failure intensity ratio.

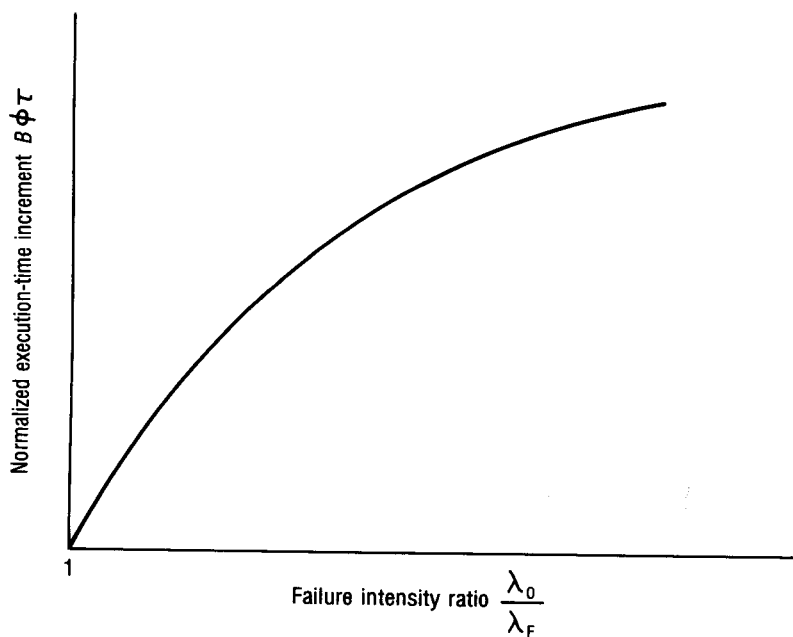


FIG. 19. Variation of normalized execution-time increment with failure intensity ratio.

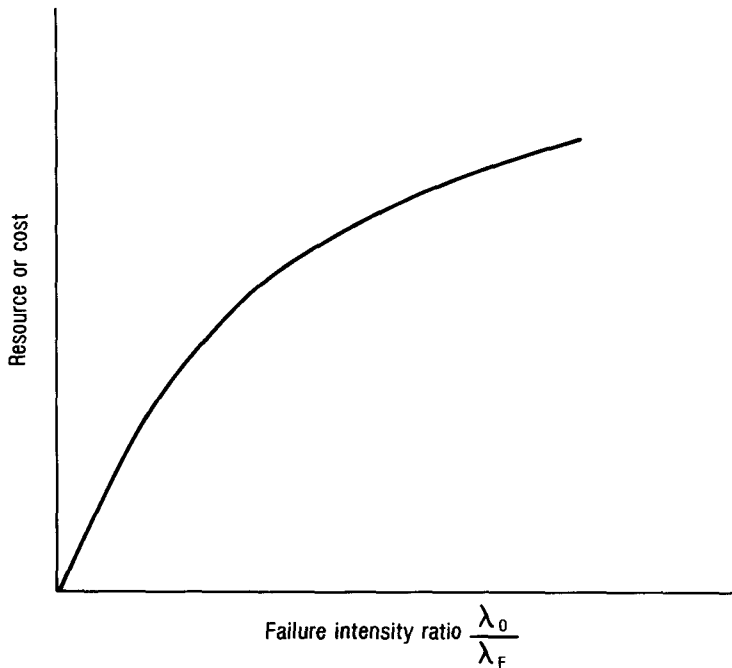


FIG. 20. General shape of failure-identification effort, computer time, and cost curves as functions of failure intensity ratio.

more stringent goal will be less than twice as much. This might seem counterintuitive. However, with the basic model, the removal of “later” faults in a system reduces failure intensity by a larger factor than removal of “early” faults. The reduction factor is $\omega/(\omega - 1)$, where ω is the number of faults remaining. Hence, although the execution time between failures and consequent costs are increasing rapidly, the preceding countervailing factor makes the relationship between cost and failure intensity sublinear.

With the foregoing approach, we implicitly assume that idle time for all resources during the project is profitably employed in other activities and is not charged as a cost. If this is not true for any resource, the cost for that resource is determined by multiplying t from Equation (3.1) by the total number of personnel (or dedicated computers) and the resource rate.

3.1.3 Operational Cost of Failures

The first step in computing reliability-dependent cost for the operational period is to establish the cost of each failure. Potential cost factors include the

cost of disrupted service on operations, the cost of restoring the system to operation, the impact of lost customers, and the cost of possible product liability actions. Restoration involves activities such as

1. making a service call,
2. terminating the unsatisfactorily operating program in an orderly fashion,
3. rebuilding corrupted databases,
4. reloading and restarting the program, and
5. documenting the circumstances of failure.

Ordinarily the fault or faults that caused the failure are not repaired in the field but are corrected in the next release of the software. However, in the case of critical failures or critical systems, field repairs may be undertaken. If so, the costs of fault detection and correction at the central site *and* distribution and installation of the correction must be added to operational-period costs. One usually assumes that the cost per failure is independent of failure intensity. However, the cost can increase nonlinearly when failure intensities are large due to increased costs of restoration when closely spaced failures interact with each other. Another source of nonlinearity is the impact of lost customers, as customer defections may increase substantially after some tolerance level is exceeded.

The total operational hours for a release is found by multiplying the number of systems installed by the expected life in operational hours for each system. To obtain the total operational cost of failures for the release, multiply the operational hours by the failure intensity achieved and cost per failure. We have assumed that failure intensity per CPU hour is constant over the life of the release. This is usually true because ordinarily there are no substantial changes (enhancements or repairs) to a program between releases. Sometimes reported failures per calendar period (day, week, etc.) vary with time, but this is due to variations in operating time and number of systems installed.

The preceding discussion focused on software failure intensity and software-failure-caused costs. However, it is also possible to take a broader view, based on computer *system* failure intensity and costs resulting from all aspects of system failure.

3.2 System Engineering

Some common system engineering applications are

1. specifying reliability to a designer,
2. estimating costs of failure for an operational system,
3. pricing a service,
4. helping determine the market window for a software-based product,

5. exploring tradeoffs among reliability and resources, cost, and schedules,
6. choosing the failure intensity objective, and
7. estimating the amount of system testing needed.

Specification of reliability may be used internally within an organization or externally with, for example, subcontractors. It can even be employed as a legal contractual requirement. An example specification might be that the reliability of a program being delivered must be 0.99 for 10 CPU hours of execution with a specified operational profile.

Costs of failure include restoration costs as noted in the previous section, but service interruptions that cause lost revenue and possibly lost customers may be even more important.

Consider the example of a system with a failure intensity of 0.025 failure per CPU hour that runs 40 CPU hours per week and requires an average of four person hours per failure to do restoration (including travel) at a cost of \$25 per hour loaded salary. Each system will average one failure per week, each requiring four hours of work and costing \$100. That is \$400 per month per system.

To price a service, we first compute failure costs as above. Then other costs and an appropriate rate of return are added to determine an initial price. Market conditions and marketing strategies must now be applied to adjust this price as necessary.

Failure intensity estimates can help determine the market window for a software-based product. The market window is the period within which a new product can be profitably introduced. The main concern is introducing a product too late, so that the competition may have already captured most of the market. Development costs can be written off early, and they then can reduce prices because of their volume. It is extremely difficult to compete in this situation. For software-based products there is likely to be a tradeoff between the time of introduction of a product and its reliability. If a product is introduced too early, it may be unreliable and gain a bad reputation, and hence not sell well.

To illustrate the use of software reliability measurement to make tradeoffs between reliability and delivery date, suppose your competition is expected to introduce a product on June 17. You use software reliability modeling and determine that if you beat out the competition by introducing a product on May 15, that is, somewhat more than a month ahead, you will have a failure intensity of 0.02 failure per CPU hour. If you decide to compete on reliability and introduce the product at the same time as your competitor, the model indicates that you will have a failure intensity of 0.001 failure per CPU hour. This reliability improvement comes from testing. The option to be selected

depends on your assessment of the market. Use of modeling helps clarify the alternatives.

Investigating tradeoffs among reliability, schedules, resources, and cost involves the following general procedure. Note that increasing reliability is equivalent to reducing the failure intensity objective. Choose several values of each parameter that will be varied. Apply an execution-time model to calculate the effects of these parameter variations. Examine the results and repeat the procedure as needed. Try to include extreme values that are reasonably possible, so that the influence of the parameters can be quickly established. Reliability and failure intensity will vary as the average instruction execution rate of the machine on which the program is running varies. Thus, one may view function and performance as additional factors when making a tradeoff. For example, a supermarket online transaction processing program A, developed for machine A, may have half the failure intensity of program B, developed for machine B. Now assume that machine B runs five times as fast. The failure intensity with respect to transactions is 2.5 times better for program B.

One may develop an initial project schedule for the system test period *before testing begins* by use of the equations presented in Section 3.1.1. The number of failures that must be experienced and the associated execution time required to attain the failure intensity objective can be predicted. The equations of Section 3.1.2 can then be used to determine system test cost. The accuracy of these *pretest* predictions depends on how well the parameters can be predicted. At present, limited data from a few projects is available for answering this question. More data is required if we are to get accurate results in the future. However, pretest predictions can be quite useful for *relative* studies.

The execution-time component parameters can be refined by *re-estimation* during test. Hence the derived quantities present failure intensity, additional number of failures that must be experienced to reach the failure intensity objective, and the associated additional execution and calendar times and completion date can also be refined.

You can also compute resource requirements and costs, giving you the capability to revisit reliability-schedule-resource-cost tradeoffs at any time during testing.

The first question to be asked in setting a failure intensity objective is, *Who* is the customer? The second is, What does the customer want? And third, Which objective is best (in the broadest sense) for you, in your competitive environment? The approach taken for a delivered product with a rapid rate of technological obsolescence may differ from that for a stable product delivered internally within a company for supplying external service. For example, if

you use a cost-based approach in setting the objective, you must know “costs to whom?”

There are at least four principal methods that can be used to establish the failure intensity objective for a system:

1. system balance,
2. release date,
3. life cycle cost optimization, and
4. marketing study.

The system-balance approach is primarily used to allocate reliabilities among components of a system. It is useful for systems in which the development is especially challenging, either because the system uses “state of the art” technology or because the reliability requirements are very demanding. The approach is often applicable to military systems. The approach *balances* the overall difficulty of development work on different components. The components with the most severe functional requirements or greatest technological risks are assigned less stringent reliability requirements. For software, functions never implemented before or based on untried algorithms would probably fall into this category. Components that are relatively standard and used in common practice would be assigned more stringent reliability requirements. This might include, for software, previously tested routines that are being reused.

The release-date approach is used when product release date is particularly critical. Schedule criticality may result from military requirements or competitive influences. Release date is fixed and you establish the failure intensity objective as a function of the resources and funds available.

For the release-date approach, one wants to have some sense of how failure intensity trades off with release date. The tradeoff of failure intensity with execution time is determined first and then a conversion to dates is made. Failure intensity at the start of system test is established by the nature of the project and the programming environment. The relationship between the ratio of failure intensity change in system test λ_0/λ_F and execution time τ for the basic execution time model is

$$\tau = \frac{v_0}{\lambda_0} \ln \left(\frac{\lambda_0}{\lambda_F} \right). \quad (3.9)$$

Since the quantities v_0 and λ_0 are fixed in any given situation, the variation of execution time τ with failure intensity ratio λ_0/λ_F will be as shown in Fig. 21.

The exact relationship of calendar time to execution time depends on the values of the calendar-time component parameters. However, the general shape is shown in Fig. 22. If Figs. 21. and 22 are combined, one obtains the

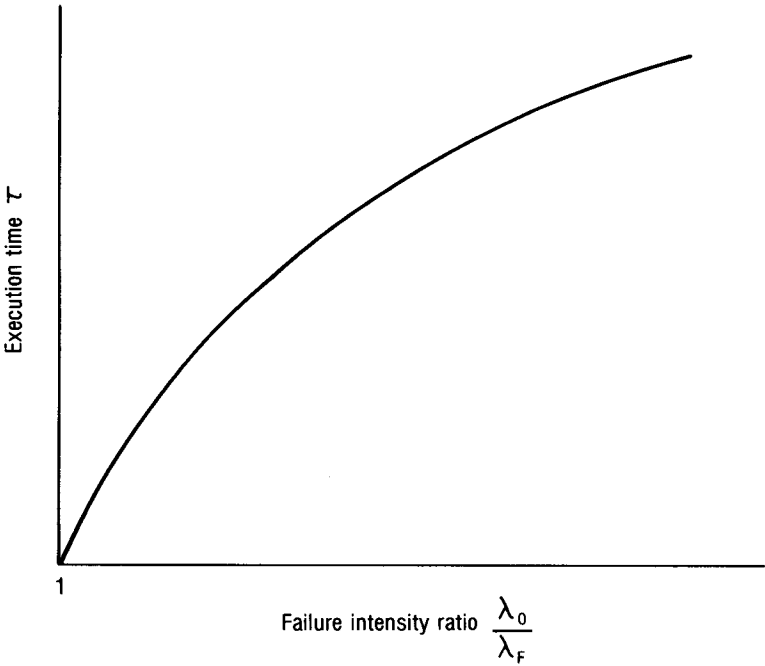


FIG. 21. Variation of execution time with failure intensity ratio.

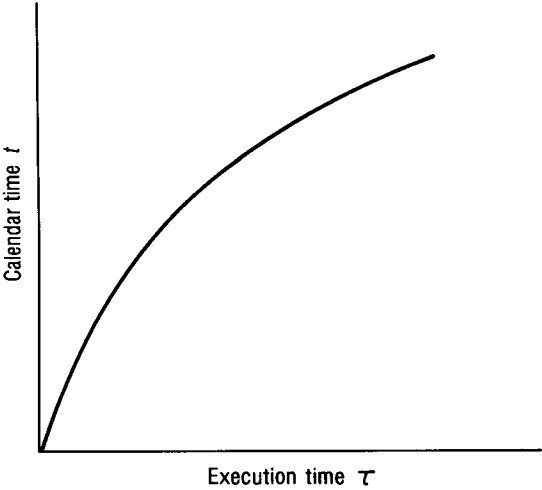


FIG. 22. Instantaneous ratio of calendar time to execution time.

general form of the relationship of calendar time with failure intensity ratio, shown in Fig. 23. The relationship can be used to determine the failure intensity ratio and hence the objective, given the release date and the resources available.

The life-cycle-cost-optimization approach especially applies to the software component of a system, but it can also readily be adapted to the overall system. It is based on reliability improvement that is obtained by more extensive testing, which of course affects costs and schedules. Costs and schedules for other phases are assumed to be constant. For any given project, reliability improvement techniques such as structured programming and design reviews are usually implemented on a "yes-no" basis dependent on their cost effectiveness. One does not usually trade off the degree to which structured programming is employed with reliability. The part of development cost resulting from testing decreases with higher failure intensity objectives, while operational cost increases (Fig. 24). Thus total cost has a minimum.

To find this minimum, first compute the system test cost (see Section 3.1.2).

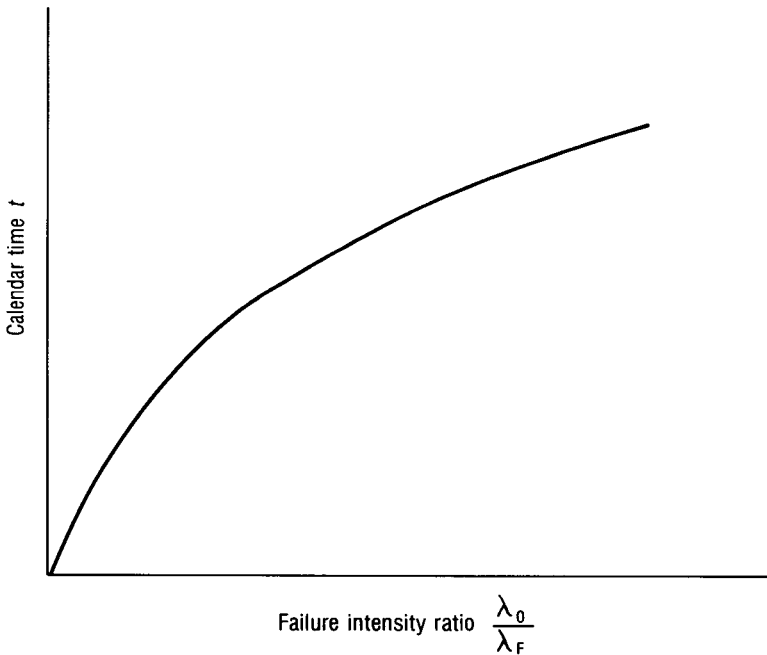


FIG. 23. Variation of calendar time with failure intensity ratio.

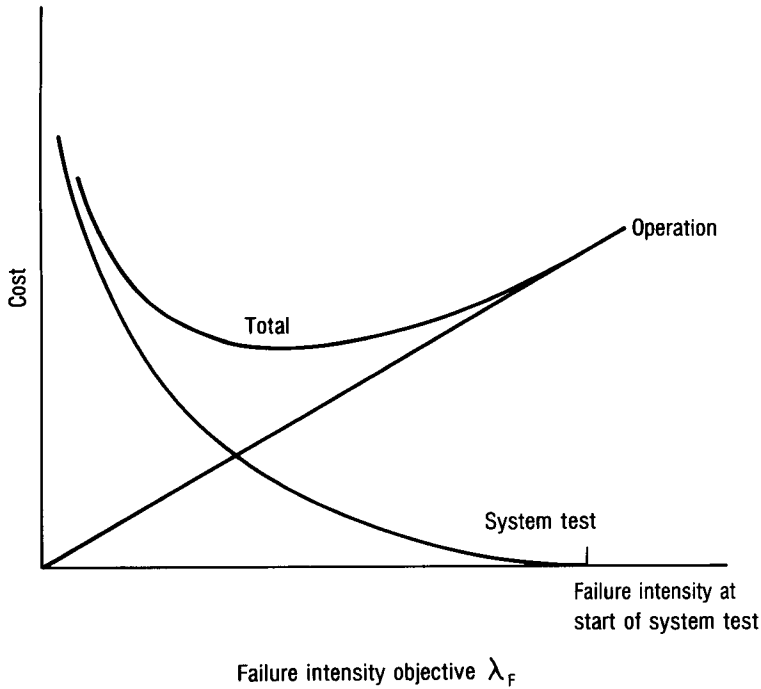


FIG. 24. Life cycle cost optimization.

Operational cost is then computed as noted in Section 3.1.3. System test and operational costs of failures are summed to obtain the *reliability-dependent component* of total system life cycle cost (there are also development and operational costs that are independent of reliability but that add to total system life cycle costs). If the reliability-related component of total system life cycle cost can be expressed analytically as a function of the failure intensity objective λ_F , then one can find the value of λ_F that minimizes the expression with calculus. If not, a computerized search process can be used.

The marketing-study approach bases the setting of the failure intensity objective on information gathered about customers and potential customers. The marketing study can take many forms, including surveys, focus groups, contacts with the customer's customers, and analysis of the customer's business. One frequently useful technique is to ask customers about any unsatisfactory experiences (failures) they had with the previous system (if such exists) and how often they occurred. Care must be taken to make the customer aware of the cost and delivery date implications of the needs expressed.

3.3 Project Management

Some common project management applications include progress monitoring, determining when a program will be ready for release, and investigation of alternatives. These applications occur when the program is being executed and at least some repair action is being taken on the failures. Most commonly, this is the system test period.

A plot from a representative project, using the basic execution-time model, is shown in Fig. 25. The plot is a project history of present failure intensity that shows progress toward the failure intensity objective. The solid center curve is the maximum-likelihood estimate. The dashed outer curves present the bounds of the 75% confidence interval. The report of Fig. 26 could have been used to evaluate progress, but the plot (Fig. 25) provides additional information for making managerial judgments.

A program is considered ready for release when the failure intensity objective will be met as the result of repair actions decreasing the failure intensity. The date is shown on the completion-date line in the tabular report in Fig. 26: Jan. 29, with a 75% confidence interval of Jan. 16 to Feb. 15. Figure 27 is a plot of the history of project completion-date predictions.

The investigation of alternatives is a reopening of the issue of tradeoffs made during system engineering. The methods of Section 3.2 apply, where the execution-time parameters are estimated from failure data.

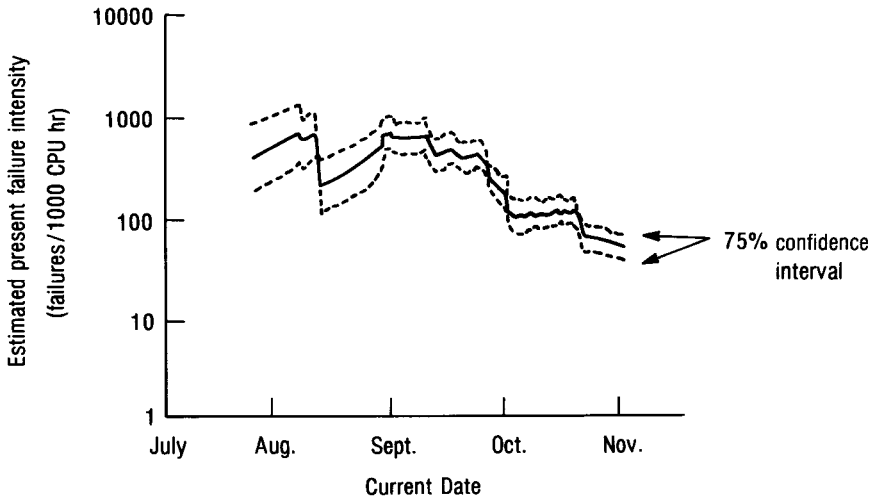


FIG. 25. History of estimates of present failure intensity.

Software Reliability Prediction—Basic Model—System T1

Based on sample of 136 test failures

Execution time is 25.34 hr

Failure intensity objective is 0.0024 failures/CPU hr

Calendar time to date is 96 days

Present date 11/9/73

	95%	Conf. Limits		50%	Most Likely	50%	Conf. Limits		95%
		90%	75%				75%	90%	
Total Failures	139	139	140	141	142	144	145	147	149
Failure Intensities (failures/1000 CPU hr)									
Initial F.I.	953	988	1044	1098	1177	1258	1315	1376	1415
Present F.I.	29	31	36	41	49	59	67	76	82
Additional Requirements to Meet Failure Intensity Objective									
Failures	3	3	4	5	6	8	9	11	13
Exec. Time (CPU hr)	34	35	38	41	46	51	56	61	65
Cal. Time (days)	42	44	47	51	57	64	69	76	80
Completion Date	10874	11074	11674	12174	12974	20774	21574	22474	30474

FIG. 26. Project status report.

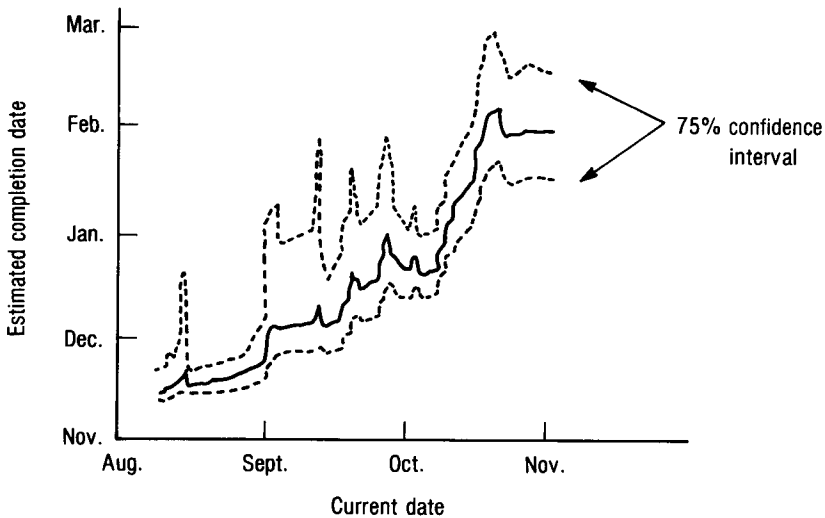


FIG. 27. History of completion date predictions.

Although software reliability measurement has primarily been applied during development to the *system* test phase, it can also be applied to subsystem testing. The subsystem size should be approximately 5000 source instructions or greater. Ultimately, it may be possible to extend the use of software reliability measurement to unit testing by grouping of units. If systems are small, estimates may have wide confidence intervals for a large part of the test period, and hence be of limited value. Further, the number of people involved is likely to be small, so that resources used in debugging may depart substantially from the average, yielding inaccurate calendar-time predictions. This can be handled by replacing average resource requirement values by individualized values obtained through measurement or estimation.

Estimates of test-period duration may be combined with estimates of the length of system requirements analysis, design, and program unit coding phases to produce overall project duration estimates. An approach to estimating time durations for either individual phases or an entire project has been developed by Putnam (1976). It is based on a Rayleigh personnel profile. Substitution of an execution-time model for the Rayleigh model in the test phases should yield better results, because more information, such as the failure intensity objective, initial failure intensity, and total expected failures, is taken into account.

Present failure intensity normally exhibits steady improvement with time during testing, although statistical fluctuations can occur. A sharp rise in failure intensity usually indicates the presence of new faults introduced by code modification or exposed by extension of testing into new environments. The latter situation may indicate poor test planning. A very slow decrease in failure intensity ordinarily indicates that many faults remain. Alternatively, one may be introducing or exposing new faults at roughly the same rate that they are being removed.

In a multi-release project, when an unreliable release is encountered, the question may arise, Should we regress to a previous version from the present one? This can be answered by tracking present failure intensity for each version. If the new version does not meet the failure intensity objective and the old one does, and if the difference between the failure intensities is substantial, it will probably be worth the effort to regress.

Tracking failure intensities of different subsystems of an overall system can be useful in helping determine the relative priorities assigned to the subsystems for failure correction. The relative criticalities of the functions implemented by the subsystems may also affect the decision.

Software reliability modeling can be used to investigate the value of overtime. Assume that we apply *selective* overtime for those debuggers with a failure backlog, and determine how this affects the calendar time required for the testing effort. Let α be the overtime fraction or ratio of overtime to

standard time. The effective available personnel are increased by the factor $(1 + \alpha)$ so that the calendar time for the failure-correction-personnel-limited segment of the test phase is reduced by that factor.

In Section 2.4.4 it was shown that the probability for each debugger that there are one or more failures awaiting correction or being corrected is ρ_F . Hence the actual fraction of overtime work for the failure-correction-personnel-limited segment is $\alpha\rho_F$. For example, suppose $\rho_F = 0.2$, $\alpha = 0.2$, and the entire test phase is failure-correction-personnel-limited. Then calendar time is reduced by a factor of $1 + \alpha = 1 + 0.2 = 1.2$. The actual overtime fraction is $\alpha\rho_F = 0.2 \times 0.2 = 0.04$. With 50% premium pay, this results in a 2% increase in cost, as the premium pay is the only extra cost. The value of *selective* overtime is clearly shown. Note the importance, however, of applying overtime only for the limiting resource, as there will be no benefit if applied to a nonlimiting resource.

A graphical method can be used to determine when (in execution time) a failure intensity objective will be reached, using the basic execution-time model. Rearranging Equation (3.9) yields

$$\ln \lambda_F = \ln \lambda_0 - \frac{\lambda_0}{v_0} \tau. \quad (3.10)$$

This relation plots as a straight line on semi-logarithmic paper. If we project the line to where it intersects the failure intensity objective, we can read off the execution time at which this occurs.

A control chart can be constructed and applied as a useful tool for management to monitor, control, and predict the quality (or the failure intensity) of a software system under test. If future failure data vary within the control limits, the process is considered to be in control. Departures from the pattern are investigated and assignable causes tracked down.

3.4 Managing the Operational Phase

3.4.1 Reliability Demonstration Tests

Frequently there is a need to prove that a given level of reliability has been achieved, especially when software is being developed by a subcontractor.

The purpose of a reliability demonstration is to establish whether a specified failure intensity objective is met with high confidence. It is assumed that an operational profile is executed that represents actual operation. Software is not repaired during the test. The reliability-demonstration approach is based on sequential sampling theory, which efficiently tests a hypothesis by taking just enough data and no more. The data points are added one by one until a decision is reached.

Figure 28 is a chart that determines whether a failure intensity objective is met within a factor (called "discrimination factor") of two. There is a 10% risk of falsely saying that the objective is not met when it is. Similarly, there is a 10% risk of falsely saying that the objective is met when it is *not*. If different discrimination factors or risk levels are desired, a different chart must be constructed (Musa *et al.*, 1987).

To conduct a reliability demonstration, first establish the failure intensity objective. Run the test and record the time of the next failure that is experienced. Normalize the failure time by multiplying it by the failure intensity objective. Now locate the normalized failure time on the reliability demonstration chart. If it falls in the "accept" region, then the failure intensity objective is met with high confidence. If it falls in the "reject" region, it is not worth testing further because there is little possibility of meeting the objective. If it falls in the "continue" region, continue to test because the situation is uncertain.

3.4.2 Impact of Design Change

Managers of operational programs often need to determine the effect of a proposed design change on the failure intensity of a program. If they can

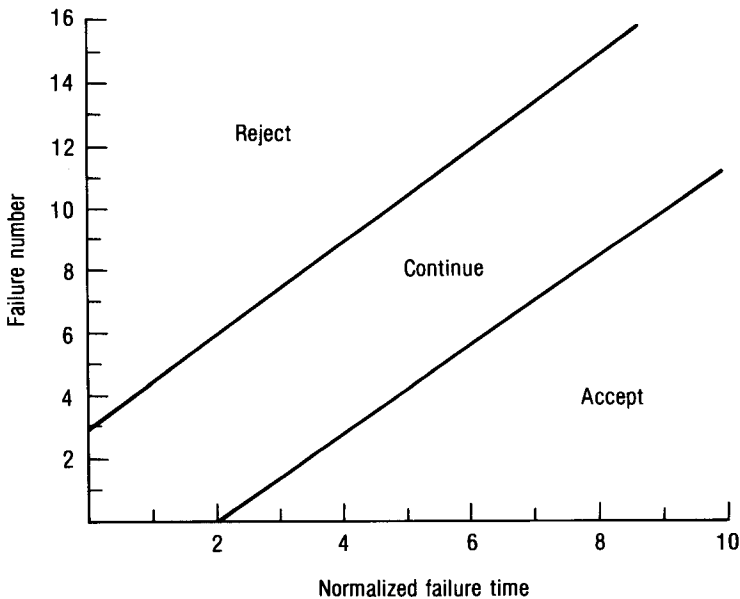


FIG. 28. Reliability demonstration chart.

estimate what the new failure intensity would be after the change, they can determine whether this failure intensity meets the service objective for the system. If it does not, they can defer the change. When the failure intensity is sufficiently low so that the proposed change does not raise the failure intensity above the service objective, it can be installed.

To predict the impact of a design change on a program, assuming the basic model, first predict the number of faults introduced by the change. This can be done by estimating the size of the change in terms of developed (new or modified) executable deliverable source instructions (or lines). Known data on average faults per source instruction (line) are applied at the appropriate point in time, usually the start of system test, as we ordinarily assume that the change has been thoroughly tested by itself before being introduced in a fashion akin to unit or subsystem test. To determine the number of faults that remain in the program at the time of the proposed change, multiply the remaining failures by the fault reduction factor B . The basic model assumes that failure intensity is proportional to the faults present. Hence the failure intensity just after the design change λ_A is predicted as

$$\frac{\lambda_A}{\lambda_B} = \frac{\omega_A}{\omega_B}. \quad (3.11)$$

Note that λ_B is the failure intensity just before the design change. The quantities ω_B and ω_A are the expected faults remaining just before and after the change, respectively.

The impacts of the design change on resource expenditures, cost, and schedules are determined by computing these quantities for both λ_B and λ_A . If product quality level is paramount, the computation is done with a fixed objective. If meeting a deadline or getting a product to market rapidly is most important, then the computation is performed using a fixed schedule.

In the preceding calculation, any effect the change might have on failure intensity by lengthening the program and hence reducing its execution frequency is considered negligible and is ignored.

The computation is based on the assumption that failure intensity is proportional to faults present. It is implicitly assumed that the code of the design change is executed as frequently as that of the existing program, on the average. Departures from this assumption occur principally for small, and hence localized, design changes. But such changes have only small impact on the program and can probably be neglected.

3.4.3 Guiding Preventive Maintenance

Many organizations, in response to failure reports, develop fixes for software in the field and then ship these fixes to the various installations.

Responses are not made to all failures, nor are fixes necessarily sent to all sites. Local managers commonly exercise considerable discretion over installation of fixes.

Software reliability concepts provide some guidance in making these decisions. Each fault has a set of failures associated with it that it can cause, and each failure results from particular run. The contribution that the fault makes to the overall failure intensity is given by the sum of the probabilities of occurrence of these different runs. Hence the failure intensity contribution of the fault depends on the operational profile.

The failure intensity contribution has an effect on the operation of the program, and each failure has some financial cost. Development, dissemination, and installation of fixes have definite financial costs also. In addition, some risk exists that installation of a fix may introduce other faults. Conducting a brief analysis of the foregoing cost impacts will help the field support manager decide on which faults to concentrate.

Adams (1984) notes that a system frequently has a relatively small number of faults that make large contributions to failure intensity. They are usually found early during the use of the system. Development and installation of fixes for these faults should be vigorously pursued. It is not necessarily best for a local site manager to install fixes on a preventive basis for all faults that have been found. It may be better to observe which failures occur frequently for the local operational profile (it may differ from the system average global one) and install corrections for just those faults that are causing those failures.

During the operational phase, failures continue to occur and new features (hence new code) continue to be added for virtually all software systems of any size. New features and fixes for discovered faults are usually introduced in scheduled releases, perhaps on the order of months apart. Then a piecewise-constant failure intensity results, each release having a different failure intensity associated with it. Sometimes fault repairs are incorporated much more frequently, perhaps weekly or more often. Then the behavior of failure intensity corresponds to that occurring during system testing.

The manager of an installation may have discretion as to when a release with new features is installed. The failure intensity of the system will exhibit a general stability about some value over the long term but with swings about this value when newly developed features are periodically incorporated. Failure intensity will increase just after each release, followed by a period of fault removal, during which the failure intensity decreases.

The field operations manager is often faced with conflicting demands. There are some users who want certain new features to be introduced as soon as possible. On the other hand, there will be people running production or other critical programs, who insist that reliability be as high as possible. These

conflicts could be negotiated and a failure intensity objective established. The manager's job is then simplified. New features are introduced only when the failure intensity is below the objective. The manager may use the amount of margin below the service objective as a guide to the size of the change to be permitted.

Figure 29 illustrates this kind of control, using data from an actual operational software system. Using the basic execution-time model, parameters were estimated and failure intensity was calculated at a number of points in time. The center line shows the most likely value of the failure intensity estimate as a function of time. The dashed lines indicate the 75% confidence bounds. A service objective of 0.05 failure per CPU hour was set. If the system is running 40 CPU hours per week, this means two failures per week. The periods of sharply rising failure intensity estimates correspond to the introduction of new features into the system. When the system is being debugged, there are slow, steady decreases. If it is decided to meet the service objective level with 75% confidence, acceptance of new features should be stopped every time the upper 75% confidence limit exceeds that level. Every time it drops below, one can consider accepting new features again. One probably wants the failure intensity estimate to drop an amount equal to the expected increment caused by the changes before they are accepted. Estimation of this increment was considered in Section 3.4.2.

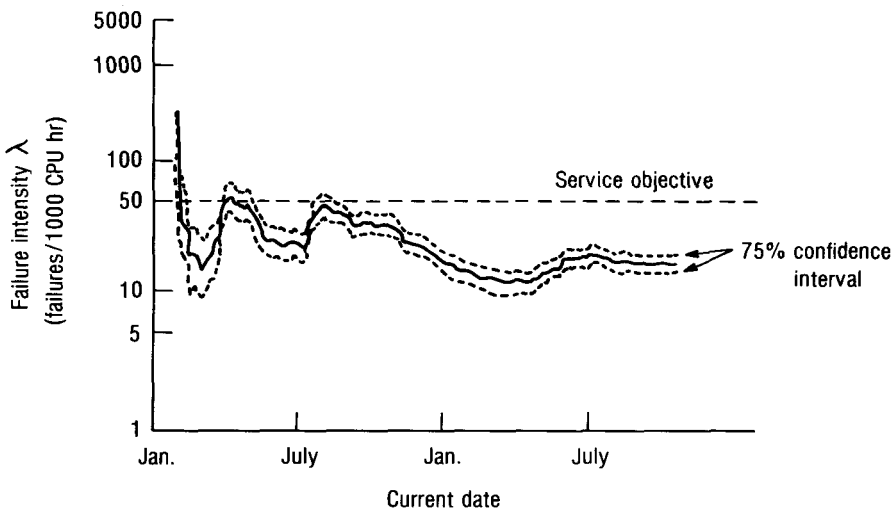


FIG. 29. Failure intensity plot used for change management of operational software.

3.5 Evaluating Software Engineering Technologies

The only way to insure that tools, methods, or technologies suggested for improving software reliability actually do so is to measure the results. One wants to relate this improvement to the cost, so it can be determined if it is worth it. To evaluate a new software engineering technology, one must first define the "old" or "control" technology. The new technology, which must be carefully described and defined, is compared with it. What seems to be a single technique may really be a set of related techniques. If there is a complex of techniques, evaluation is riskier because all members of the complex may not always be applied with equal vigor. An appropriate experiment to compare the two technologies must be developed. Experimental design theory may help guide the experimenter toward efficient investigation. Efficiency may be important because software engineering experiments can be costly.

There are two common alternatives in making comparisons, a fixed-schedule and a fixed-objective approach. To apply the fixed-schedule approach, measure failure intensity for both experimental and the control technologies after equal amounts of test. Any improvement in failure intensity is judged against the cost of applying the technique. This method is best if the release date is constrained (for example, by marketing considerations). For the fixed-objective approach, test both cases until the same failure intensity is reached. Compute the cost saving, if any. The saving is the reduction in test costs due to shorter test period, minus the cost of applying the technique.

Effective new software engineering technologies that are applied during phases before testing usually reduce the number of inherent faults. Assume the basic model. Reducing inherent faults by 10% (for example) will reduce $v_0 (= \omega_0/B)$, the total number of failures in infinite time, by 10%. It will also decrease $\lambda_0 (= fK\omega_0)$, the initial failure intensity at the start of test, by 10%. The failure intensity curve is described by

$$\lambda = \lambda_0 \exp\left(-\frac{\lambda_0 \tau}{v_0}\right), \quad (3.12)$$

where τ is execution time. Note that the curve is lowered, the amount being 10% at $\tau = 0$.

Attaining a given failure intensity objective λ_F from the initial failure intensity λ_0 requires that $\Delta\mu$ failures must be experienced and $\Delta\tau$ of execution time expended. We have

$$\Delta\mu = v_0 \left(1 - \frac{\lambda_F}{\lambda_0}\right) \quad (3.13)$$

and

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_0}{\lambda_F}\right). \quad (3.14)$$

The new technology will reduce both $\Delta\mu$ and $\Delta\tau$ in this example, but by less than 10%. Let $\lambda_F = 0.01\lambda_0$. Then $\Delta\mu$ will be reduced almost 10% and $\Delta\tau$ by slightly over 2%. The net effect on the duration and cost of the test period will depend on the values of the set of resource and cost parameters, but it will probably fall somewhere between 2 and 10%. An alternative view is that a successful software engineering technology reduces the failure intensity ratio that must be attained over the test period, for a fixed failure intensity objective. Thus duration and cost of testing can be reduced as shown in Figs. 19 and 20.

The new technology may affect program size as well as the number of faults it contains. One example is that writing a program in structured form may lengthen it somewhat. In that case, the linear execution frequency (average instruction execution rate divided by size) is reduced. The analysis of the effect of the new technology is slightly more complicated, since it affects two parameters.

3.6 Documentation Debugging and Other Applications

It is likely that software reliability concepts, methodologies, and tools can be generally applied to products of certain systematic creative processes and to the processes themselves since software development is such a creative process. These processes are characterized by initial implementation followed by "failure" detection through systematic "test" and fault correction. Hardware design and writing are examples of such potential applications.

A draft of a paper, article, or document is "system tested" through reading by reviewers. The "system requirements" are that information be conveyed to meet particular needs, which might be diagnosing a software failure or determining how to add a change. Test "runs" are attempts to obtain information to meet these needs. The "operational profile" describes the range and relative frequencies of these needs. Any particular run will occur randomly with a probability expressed by the operational profile. A "failure" occurs when reading the text does not convey the information required to meet the need. The fault is the text deficiency that is responsible for the failure. The failure intensity decreases with time as the document is "repaired." One can apply software reliability models to this situation. Establish an objective for the document such as "0.95 'reliability' in obtaining the required information for various needs per hour of reading." This approach may be useful in

evaluating user manuals and program documentation, and it might conceivably also be applied to the evaluation of requirements or design specifications.

4. State of the Art, Future Development, and Summary

In summary, the field of software reliability measurement and prediction has made substantial progress in the last decade. There are several problem areas that need work. However, the technology is clearly beyond the pure theory state and it can provide definite practical dividends for those making the modest investment in time needed to learn and apply it. Each of the main application areas for software reliability measurement and prediction will be considered with regard to the state of the art and areas needing further research.

4.1 Measurement

Software reliability can now be measured in the operational environment with excellent accuracy. Hence, use of failure intensity as a guide to control change in operational systems or as a way to evaluate software engineering technologies is currently feasible (Musa, 1979b).

The quality of software reliability measurement based on testing depends on the accuracy with which test and operational execution environments are known. If one cannot represent the operational environment satisfactorily, then one must compensate for the difference between it and the test environment. There is also a need to be able to estimate the change in failure intensity that would result from a changing operational environment. Finally, there is a natural difference between the test and operational environments that results from eliminating redundancy in tests. Methods for dealing with these situations have been developed (Musa *et al.*, 1987), but they need to be more widely applied and tuned as experience is gained in their use. Based on the current capability to measure software reliability during test, status monitoring and tracking of projects can be accomplished with a relatively good level of quality.

4.2 Prediction Quality and Its Improvement

The quality of prediction of software reliability and other quantities associated with failure behavior is very important, as many significant managerial and engineering decisions depend on it.

4.2.1 Parameter Estimation

The current quality of prediction of failure behavior in execution time based on parameter estimation, which requires that the program be executed and failure data collected, could be characterized as good. Although the accuracy may be sufficient for many purposes, prediction often can be biased. Finite-failures category models tend to be optimistic in that they underpredict the number of failures to be experienced by a future point in time. Infinite-failures category models tend to be neutral or pessimistic. There is a fair amount of dispersion in the predictions (Musa *et al.*, 1987) in both cases. Techniques that could reduce the prediction bias and/or the dispersion would clearly be beneficial. Improvement in predicting failure intensity would improve predictions of the dates for reaching various failure intensity objectives. Such predictions are valuable from project schedule and product marketing perspectives. Two general strategies have been suggested to improve the situation: improving parameter estimation, and improving prediction directly.

Joe and Reid (1985) studied the estimation problem for a certain kind of binomial model and observed that it is surprisingly difficult to find an estimator that is unbiased in all situations. For values of total failures between 10 and 20, the harmonic mean of the endpoints of the 50% likelihood interval for total failures was the most effective estimator. However, further investigation may still lead to improvements in inference in determining model parameters and their confidence intervals.

Current approaches to directly improving prediction of failure behavior (Littlewood *et al.*, 1986) concentrate either on adaptive prediction or on taking weighted averages of predictions from different models. One type of adaptive prediction may be visualized as using what Littlewood calls a “*u*-plot” as an indicator of prediction bias. A “core” model is used to predict the distribution of the failure time τ_i , given as $F(\tau_i | \tau_{i-1})$. The core model can be any of the models described in this article. First, the model parameters must be estimated from the known failure times $\tau_1, \tau_2, \dots, \tau_{i-1}$. The prediction, denoted by $\tilde{F}(\tau_i | \tau_{i-1})$, is then made, using the model with the parameter values that have just been estimated. The actual failure time τ_i is observed and transformed by the probability integral transform corresponding to the predicted distribution:

$$u_i = \tilde{F}(\tau_i | \tau_{i-1}). \quad (4.1)$$

The probability integral transformation is the hypothesized cumulative distribution function of the data being transformed (Mood *et al.*, 1974). If the hypothesis is correct, then the transformed data u_i will be distributed uniformly. Predictions are made step-by-step, starting with i at a value that gives a sample of reasonable size (perhaps 10 or 20). Thus a set of independent

u_i to be tested are developed. Note that predictions for any fixed number of failures ahead in the future can be made, not just one.

The u -plot is constructed by first ordering the u_i in ascending order (assume there are m values). Then plot a point corresponding to the first u_i on the horizontal axis and $1/(m+1)$ on the vertical axis. The axes represent, respectively, the empirical and theoretical quantiles. The next point is the coordinate of the second u_i and $2/(m+1)$. Continue for all m values. Figure 30 illustrates the plot. Here the u -values are excessively small, indicating optimistic predictions.

The prediction bias is related to the distance between the u -plot and the perfect prediction line of slope one. Before attempting any correction one must first be sure that any bias that exists is reasonably stationary by checking to see if there is any trend in the u_i data. A way to do this is to transform the u_i using

$$x_i = -\ln(1 - u_i). \quad (4.2)$$

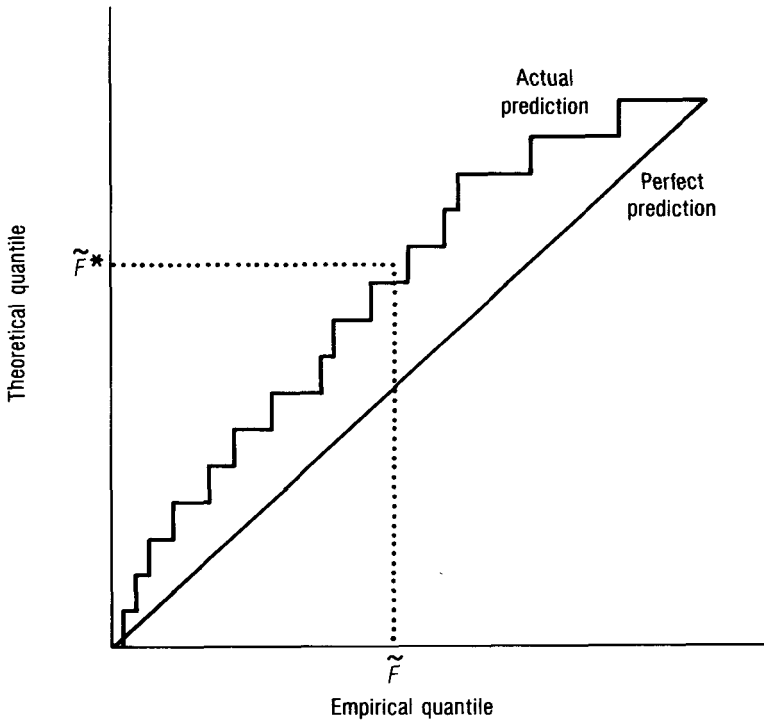


FIG. 30. Sample u -plot.

This transformation yields independent unit exponential random variables if the u_i are independent and uniform. A test for trend can be made by testing for a homogeneous Poisson process (Cox and Lewis, 1966).

Assuming that the bias is stationary, an adjusted prediction $\tilde{F}^*$ can be developed by modifying the raw prediction $\tilde{F}$ as some function of the distance between the u -plot and the perfect prediction line. Preliminary results indicate that this type of adaptive prediction removes much of the prediction bias experienced with models. It may also reduce the differences in predictive validity experienced between models, which would increase the importance of other criteria in selecting a model.

The other method for improving predictive validity, taking weighted averages of predictions from different models, also appears promising. One way to determine the weights might be to take them from some estimates of the bias such as the u -plot. However, this approach may have more disadvantages than the adaptive approach. It would probably result in a loss of physical feel for the model, so that it would be difficult to relate parameters to characteristics of the software and the software development process.

4.2.2 *Parameter Prediction*

The prediction of software reliability from software product and development process characteristics requires the most work (Musa, 1979b). But, it also offers great potential benefits, since it is crucial to system engineering studies and will establish how well early planning decisions can be made. These decisions can have the greatest impact on schedules and cost. With the program not yet executable, the execution-time component parameters must be predicted from program characteristics rather than estimated from failure data. For the basic execution-time model, the number of inherent faults ω_0 , the fault reduction factor B , the fault exposure ratio K , and the linear execution frequency f are needed.

The best current approach to determining the number of faults is to use program size. Data on the relationship between faults and size is available, but more is needed. There are probably other variables that affect the number of inherent faults, and some possibilities are starting to emerge. Complexity of the program would at first appear to be one, but despite much research, no one has yet been able to capture convincingly the concept of complexity in a simple metric and demonstrate that it improves the prediction of the number of inherent faults. One needs to find a small, readily measurable set of additional factors that improve the predictability of the number of inherent faults. Also needed are better ways to estimate size of a program in the requirements stage, before any code has been written.

Initial data indicates that the fault reduction factor B may be fairly stable

across different projects, but more study is required. If it is not stable, the project variables that most effect it should be determined. The fault exposure ratio K may depend on the dynamic structure of the program and the extent to which faults depend on data values. However, these effects may average out for programs of substantial size. It is important to know the value of this parameter to obtain good absolute software reliability predictions.

Even if good absolute values for system engineering studies are unavailable, studies involving relative comparison can often be conducted with good results.

4.2.3 *Calendar Time*

Prediction in calendar time is currently fair in quality (Musa, 1979b). Calendar-time prediction necessitates knowing the values of the planned and resource usage parameters. Data on a number of projects should be collected to determine the values of the resource usage parameters and how much they vary between different projects or classes of projects. If they do vary, the factors involved should be studied.

4.3 Possible Extensions

A relatively unexplored area of possible extension of software reliability models, is the unit test phase. Failures occur in this phase and times of failure are available; consequently, it should be theoretically possible to make reliability estimates. There are two main problems for which solutions are needed:

1. small failure samples, and
2. relating unit tests to the operational profile.

The small failure sample problem might be handled by grouping together failures from the different units. The second problem appears more difficult. Unit testing is usually conducted from a structural or "white box" viewpoint. Tests are selected to maximize "coverage" of instructions, branches, or paths (best). It is hard to determine which input state is being exercised by any given unit test. Furthermore, a unit test generally represents only a partial and incomplete test of the input state. Consequently, it is difficult to determine what operational profile has been exercised by a given set of unit tests. Does the set represent the operational environment? If not, we'll need a way to make appropriate adjustments to failure intensities.

As software reliability models are applied to actual projects, a collection of techniques for handling various special situations will be developed and some

existing techniques may be refined. We need to know how different projects classify failure severity, and the proportions of failures that fall in each class. Can we verify with more certainty that the proportions tend to be stationary? When a system is undergoing substantial program design change, would it be beneficial to limit the range of data used for estimation of parameters?

It would be very useful if there were a single reliability-related metric that could actually be *measured*, rather than just predicted, through all phases of the software life cycle. This is a difficult problem, because failures appear to be the only entities that are closely related to reliability. Faults in code have some relationship with reliability, but the operational profile also has a strong influence. We could define design and requirements errors by generalizing the concept of faults in code. However, such faults are even more "distant" in their relationship with reliability because additional factors may intervene. Furthermore, faults cannot be tracked against the same base, execution time, as failures. One might track them against some quantity such as inspection effort (in person hours) but then you create the problem of different independent variables. Yet at present, errors and faults are quantities that are highly observable in the phases before execution. For example, requirements and design errors are detectable through such processes as requirements reviews and design errors are detectable through such processes as requirements reviews and design inspections. Code faults can be found through code reading, either by desk checks or code walkthroughs.

The most promising approach might be to use failures for all project phases. Failures would then have to be defined in terms of simulated execution during the early development phases. This implies executable requirements and design languages, or a way of determining the fail set (input space) of every fault that is identified. Furthermore, one would have to develop a good way of estimating execution time from the "time" recorded for simulated executions. But, if these problems could be solved, one would have the mechanism for looking at the entire software life cycle and allocating reliability improvement goals to each phase. Then "reliability risk" and challenge could be spread rationally across the phases, perhaps so as to minimize reliability improvement cost.

Some software reliability techniques are applicable to hardware design faults. Ascher and Feingold (1982) recommended further investigating this possibility. This article has dealt with the question of reliability of programs, but reliability of databases is becoming an increasingly important question. Little is known about this subject. However, artificial intelligence applications (particularly expert systems) are expected to increase greatly the impact of database reliability (or the lack of it). Database faults, unlike program faults, result from both structural errors (misunderstanding the system requirements for the definition of the database) and data entry faults.

There have been at least two interesting side benefits of software reliability modeling. The conceptual framework provided by a good model yields greater insight into the software failure process. The processes of defining just what constitutes a failure and of setting a failure intensity objective have both been salutary in stimulating communication between the customer and the developer.

REFERENCES

- Adams, E. N. (1984). Optimizing Preventive Service of Software Products. *IBM J. Research and Development* **28** (1), 2–14.
- Ascher, H. E., and H. Feingold. (1982). Repairable Systems Reliability: Future Research Topics. In "Reliability in Electrical and Electronic Components and Systems" (E. Langer and J. Moltoft, eds.), pp. 81–88. North-Holland, Amsterdam.
- Cinlar, E. (1975). "Introduction to Stochastic Processes." Prentice-Hall, Englewood Cliffs, New Jersey.
- Cox, P. R. (1981). Specification of a Regression Test for a Mini Computer Operating System. *Performance Evaluation Review* **10** (1), 29.
- Cox, D. R., and Lewis, P. A. W. (1966). "Statistical Analysis of Series of Events." Methuen, London.
- Crow, L. H. (1974). Reliability Analysis for Complex, Repairable Systems, In "Reliability and Biometry" (F. Proshan and R. J. Serfling, eds.), pp. 379–410. SIAM, Philadelphia.
- Dale, C. J. (1982). Software Reliability Evaluation Methods. British Aerospace, Report ST26750.
- Ehrlich, W. K., and Ejzak, R. (1988). Approximating Program Execution Time for the Purpose of Software Reliability Modeling. *Proc. Fourth Annual National Conf. Software Quality and Productivity*.
- Goel, A. L., and Okumoto, K. (1978). An Analysis of Recurrent Software Errors in a Real-time Control System. *Proc. ACM Conf.*, pp. 496–501.
- Goel, A. L., and Okumoto, K. (1979a). Time-Dependent Error-Detection Rate Model for Software Reliability and other Performance Measures. *IEEE Trans. Reliability* **R-38** (3), 206–211.
- Goel, A. L., and Okumoto, K. (1979b). A Markovian Model for Reliability and other Performance Measures. *Proc. National Chapter Conf.*, pp. 769–774.
- Hecht, H. (1977). Measurement, Estimation, and Prediction of Software Reliability. In "Software Engineering Technology," Volume 2, pp. 209–224. Infotech International, Maidenhead, Berkshire, England.
- Hecht, H. (1981). Allocation of Resources for Software Reliability. *Proc. COMPCON Fall (1981)*, pp. 74–82.
- Herndon, M. A., and Keenan, N. T. (1978). Analysis of Error Remediation Expenditures During Validation. *Proc. Third Int. Conf. Software Engineering*, pp. 202–206.
- Hudson, G. R. (1967). Program Errors as a Birth and Death Process. System Development Corporation, Rep. SP-3011, Santa Monica, California.
- Iannino, A., Musa, J. D., Okumoto, K., and Littlewood, B. (1984). Criteria for Software Reliability Model Comparisons. *IEEE Trans. Software Engineering* **SE-10** (6), 687–691.
- Jelinski, Z., and Moranda, P. B. (1972). Software Reliability Research. In "Statistical Computer Performance Evaluation" (W. Freiberger, ed.), pp. 465–484. Academic Press, New York.
- Joe, H., and Reid, N. (1985). Estimating the Number of Faults in a System. *J. American Stat. Assoc.* **80** (389), 222–226.
- Keiller, P. A., Littlewood, B., Miller, D. R., and Sofer, A. (1983). On the Quality of Software Reliability Prediction. In "Electronic Systems Effectiveness and Life Cycle Costing" (J. W.

- Skwirzynski, ed.), pp. 441–460. NATO ASI Series, F3, Springer-Verlag, Heidelberg.
- Kleinrock, L. (1975). "Queueing Systems," Vol. 1. Wiley, New York.
- Kremer, W. (1983). Birth-Death and Bug Counting. *IEEE Trans. Reliability* **R-32** (1), 37–47.
- Langberg, N., and Singpurwalla, N. (1985). A Unification of Some Software Reliability Models. *SIAM J. Scientific and Statistical Computing* **6** (3), 781–790.
- Littlewood, B. (1981). Stochastic Reliability-Growth: A Model for Fault-Removal in Computer-Programs and Hardware-Design. *IEEE Trans. Reliability* **R-30** (4), 313–320.
- Littlewood, B., and Verrall, J. L. (1973). A Bayesian Reliability Growth Model for Computer Software. *J. Royal Stat. Soc., Series C*, **22** (3), 332–346.
- Littlewood, B., Ghaly, A. A., and Chan, P. Y. (1986). Evaluation of Competing Software Reliability Predictions. *Proc. NATO Advanced Study Institute*, Durham, U.K., July 29–Aug. 10, 1985.
- Lloyd, D. K., and Lipow, M. (1977). "Reliability: Management, Methods, and Mathematics," second edition. Published by the authors, Redondo Beach, California.
- Miller, D. R. (1986). Exponential Order Statistic Models of Software Reliability Growth. *IEEE Trans. Software Engineering* **12** (1), 12–24.
- Mood, A. M., Graybill, F. A. and Boes, D. C. (1974). "Introduction to the Theory of Statistics." McGraw-Hill, New York.
- Moranda, P. B. (1975). Predictions of Software Reliability During Debugging. *Proc. Annual Reliability and Maintainability Symp.*, pp. 327–332.
- Musa, J. D. (1975). A Theory of Software Reliability and its Application. *IEEE Trans. Software Engineering* **SE-1** (3), 312–327.
- Musa, J. D. (1979a). Software Reliability Data. Report available from Data and Analysis Center for Software, Rome Air Development Center, Rome, New York.
- Musa, J. D. (1979b). Software Reliability Modeling—Where Are We and Where Should We be Going? *Proc. Fourth NASA Software Engineering Workshop*, pp. 239–249.
- Musa, J. D. (1979c). Validity of Execution Time Theory of Software Reliability. *IEEE Trans. Reliability* **R-28** (3), 181–191.
- Musa, J. D. (1985). Software Engineering: The Future of a Profession. *IEEE Software* **2** (1), 55–62.
- Musa, J. D., and Okumoto, K. (1983). Software Reliability Models: Concepts, Classification, Comparisons, and Practice. In "Electronic Systems Effectiveness and Life Cycle Costing (J. K. Skwirzynski, ed.), pp. 395–424. NATO ASI Series, F3, Springer-Verlag, Heidelberg.
- Musa, J. D., and Okumoto, K. (1984a). A Logarithmic Poisson Execution Time Model for Software Reliability Measurement. *Proc. Seventh Int. Conf. Software Engineering*, pp. 230–238.
- Musa, J. D., and Okumoto, K. (1984b). A Comparison of Time Domains for Software Reliability Models. *J. Systems Software* **4** (4), 277–287.
- Musa, J. D., Iannino, A., and Okumoto, K. (1987). "Software Reliability: Measurement, Prediction, Application." McGraw-Hill, New York.
- Myers, G. J. (1979). "The Art of Software Testing." Wiley, New York.
- Putnam, L. H. (1976). A Macro-Estimating Methodology for Software Development. *Digest of Papers: Fall COMPCON (1976)*, pp. 138–143.
- Ramamoorthy, C. V., and Bastani, F. B. (1982). Software Reliability—Status and Perspectives. *IEEE Trans. Software Engineering* **SE-8** (4), 354–370.
- Ramamoorthy, C. V., Prakash, A., Garg, V., Yamaura, T., and Bhide, A. (1987). Issues in the Development of Large, Distributed, and Reliable Software. In "Advances in Computers" 26 (M. C. Yovits, Ed.), pp. 393–343. Academic Press, New York.
- Saunier, P. (1983). Fiabilité du logiciel: quelques remarques tirées d'une expérimentation de trois modèles à taux de panne, (in French). *L'Industrie Face à la Qualité du Logiciel*, Belvedere, Toulouse, France, Oct. 20–21, 1983, pp. 257–290.
- Schick, G. J., and Wolverton, R. W. (1973). Assessment of Software Reliability. *Proc. Operations Research*, pp. 395–422.

- Schick, G. J., and Wolverton, R. W. (1978). An Analysis of Competing Software Reliability Models. *IEEE Trans. Software Engineering* **SE-4** (2), 104–120.
- Schneidewind, N. F. (1972). An Approach to Software Reliability Prediction and Quality Control. *1972 Fall Joint Computer Conf. AFIPS Conf. Proc.* **41**, pp. 837–847.
- Schneidewind, N. F. (1975). Analysis of Error Processes in Computer Software. *Proc. 1975 Int. Conf. Reliable Software*, pp. 337–346.
- Shanthikumar, J. G. (1981). “A State- and Time-Dependent Error Occurrence-Rate Software Reliability Model with Imperfect Debugging. *Proc. National Computer Conf.*, pp. 311–315.
- Shooman, M. L. (1972). Probabilistic Models for Software Reliability Prediction. In “Statistical Computer Performance Evaluation” (W. Freidberger, ed.), pp. 485–502. Academic Press, New York.
- Shooman, M. L. (1984). Software Reliability: A Historical Perspective. *IEEE Trans. Reliability* **R-33** (1), 48–55.
- Shooman, M. L. (1986). “Probabilistic Reliability: An Engineering Approach.” McGraw-Hill, New York, (1968). Updated and reprinted, Krieger, Malabar, Florida.
- Shooman, M. L., and Bolsky, M. I. (1975). Types, Distribution, and Test and Correction Times for Programming Errors. *Proc. 1975 Int. Conf. Reliable Software*, pp. 347–357.
- Sukert, A. N. (1979). Empirical Validation of Three Software Error Prediction Models. *IEEE Trans. Reliability* **R-28** (3), 199–205.
- Trachtenberg, M. (1985). The Linear Software Reliability Model and Uniform Testing. *IEEE Trans. Reliability* **R-34** (1), 8–16.
- Wagoner, W. L. (1973). The Final Report on a Software Reliability Measurement Study. Aerospace Corporation, Report TOR-0074(4112)-1.
- Yamada, S., Ohba, M., and Osaki, S. (1983). S-Shaped Reliability Growth Modeling for Software Error Detection. *IEEE Trans. Reliability* **R-32** (5), 475–478.